

AUM SRI SAI RAM

SRI SATHYA SAI HIGHER SECONDARY SCHOOL

PRASANTHI NILAYAM



INFORMATICS PRACTICES

PROJECT REPORT

FOOD ORDERING SYSTEM

CRAVING CAFE

WHERE FOOD MEETS JOY

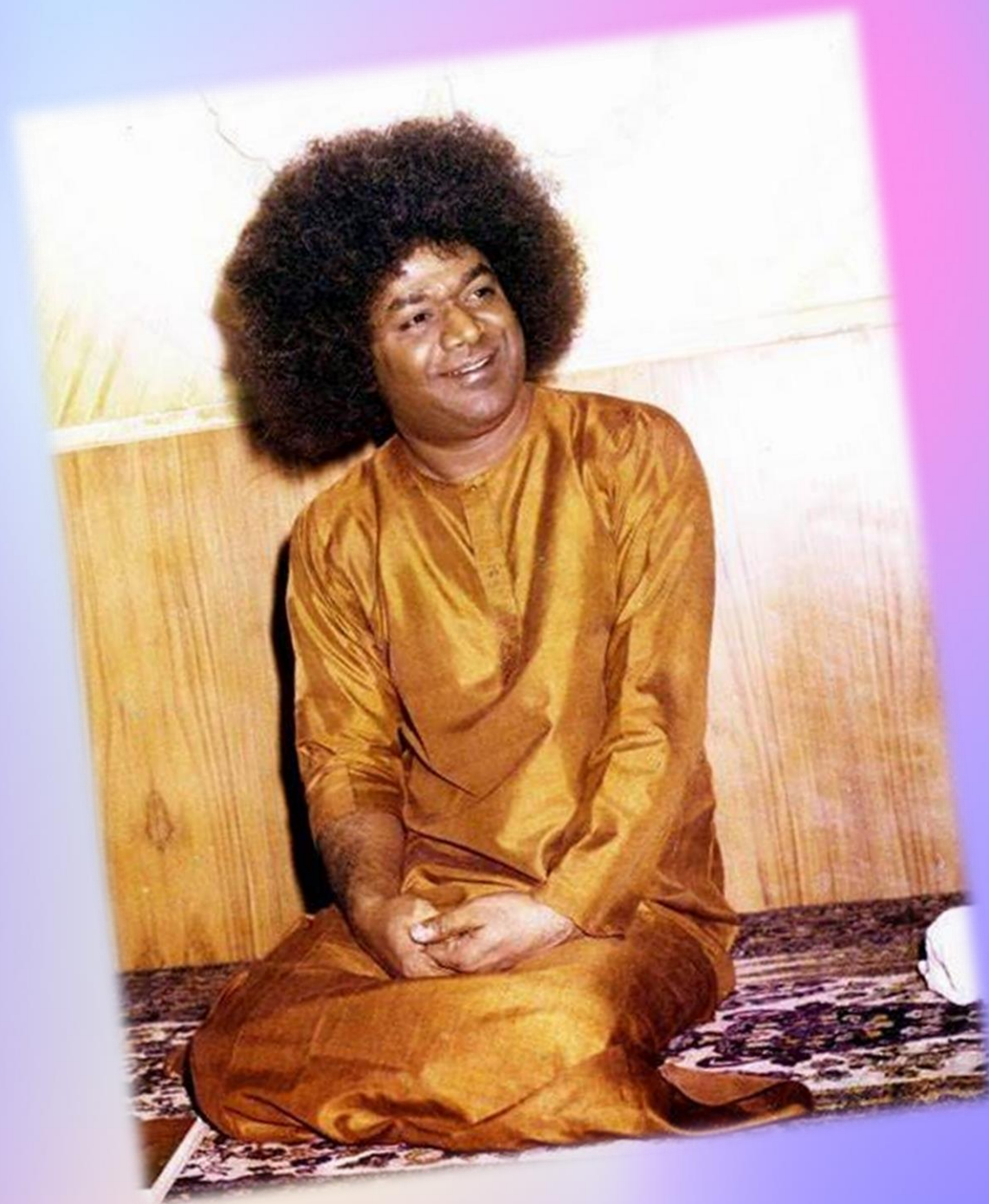
NAME

Saidarshan Behera

CLASS

XII MAE Info

ROLL NO.



**DEDICATED AT THY LOTUS
FEET**

SRI SATHYA SAI HIGHER SECONDARY SCHOOL
PRASHANTHI NILAYAM



CERTIFICATE

This is to certify that the Informatics Practices
project entitled FOOD ORDERING SYSTEM
submitted by Master SAIDARSHAN BEHERA
having roll no of class 12th has completed this project
for partial fulfillment of the requirement for the Senior Secondary
Examination (AISSE) embodies the bonafide work done by him
under my supervision.

INTERNAL SUPERVISOR

Signature..

Name

Date

PRINCIPAL

Signature..

Name

Date

EXTERNAL SUPERVISOR

Signature..

Name

Date

ACKNOWLEDGMENT

I extend my heartfelt gratitude to everyone who has generously and sincerely contributed to the success of this project. At the core of every successful endeavor lies the guiding hand of the almighty. To **BHAGAWAN SRI SATHYA SAI BABA**, I offer my deep sense of gratitude. I am also thankful for the dedicated teachers he has bestowed upon us.

This project has benefited from the support of many individuals. I am immensely grateful to each and every one of them for helping me bring this extraordinary project to fruition. I extend my sincere thanks to the school faculty for their unwavering support and guidance, which has been a crucial element in the completion of this project.

I am particularly grateful to our esteemed principal, **SRI Y SHIVARAMAKRISHNAIAH SIR**, and to the teacher, **SRI P SAI PAVAN SIR**, for his boundless support and patience. His extensive knowledge and passion for the subject have been incredibly beneficial throughout the course of this project. His problem-solving skills and dedication have made him an indispensable mentor in the completion of this endeavor.

Lastly, my heartfelt thanks go to my fellow informatics classmates for their all-around support in making this project a grand success.

TABLE OF CONTENTS

AIM.....	1
INTRODUCTION.....	2
SCOPE OF THE PROJECT.....	4
OVERVIEW	5
MODULES.....	6
SOFTWARES USED	7
LIBRARIES USED	9
SYSTEM REQUIREMENTS	11
DATABASE.....	13
PROGRAM CODE.....	15
UI SCREENSHOTS	117
FUTURE ENHANCEMENTS.....	124
BIBLIOGRAPHY.....	125

AIM

This project seeks to develop an advanced **Food Ordering System** that streamlines the ordering process for food outlets. It will enable users to place orders, manage menus, and track order history with ease. The system will also include sales analysis tools for monitoring trends and inventory, while offering a user-friendly interface and secure database to enhance operational efficiency and customer satisfaction.

INTRODUCTION

The **Food Ordering System** is designed to modernize and optimize the way food outlets manage their ordering and operational processes. By offering a seamless experience for staff and administrators, this system revolutionizes the traditional food ordering workflow, ensuring improved efficiency, accuracy, and user-friendliness.

At its core, the system empowers staff to place orders on behalf of customers, simplifying the ordering process while maintaining accuracy and efficiency. Admins have easy access to review order history and verify each request through an intuitive, user-friendly interface. While customers no longer need to place orders themselves, the system facilitates seamless communication of their preferences, ensuring a smooth and personalized ordering experience.

The admin role has broader control over the system, enabling the management of various critical functions. Administrators can manage the entire menu by adding, removing, or updating items in real time. This ensures that the menu is always current and free from errors, minimizing operational disruptions. Additionally, admins can access comprehensive sales analysis tools to gain insights into sales trends, customer preferences, and inventory levels. This data empowers them to make informed decisions about stock management, pricing strategies, and overall business planning.

Sales analysis tools are one of the standout features of the system, offering detailed insights into performance across different time periods—whether daily, monthly, or yearly. The data provides actionable insights into sales trends, inventory management, and

customer behavior, giving food outlets the ability to optimize operations and increase profitability.

Built on a robust database infrastructure, the system ensures secure and reliable storage of all data, including orders, customer information, and sales records. The database supports fast and efficient system performance, providing both staff and admins with easy access to the information they need to keep the business running smoothly.

Designed with user experience in mind, the system offers an intuitive interface for staff to manage orders and for admins to control business operations. By streamlining the order placement process, enhancing menu management, and providing in-depth sales analysis, the Advanced Food Ordering System empowers food outlets to optimize their operations, improve customer satisfaction, and drive business growth.

This system aims to set a new standard in the food service industry, creating a more dynamic, data-driven approach to food ordering and overall business management, with roles clearly defined for staff and admins to ensure smooth and effective operation.

SCOPE OF THE PROJECT

The scope of the **Food Ordering System** includes streamlining food outlet operations through an efficient order management process. It allows staff to place and track orders, while admins manage the menu, sales analysis, and inventory. The system will provide real-time order updates, secure data storage, and detailed sales reports for trend analysis. Admins will have full control over menu customization and inventory tracking. The user-friendly interface will be accessible on both mobile and desktop platforms. The system will also integrate with existing tools like POS systems and payment gateways, ensuring seamless operations and scalability for future growth.

OVERVIEW

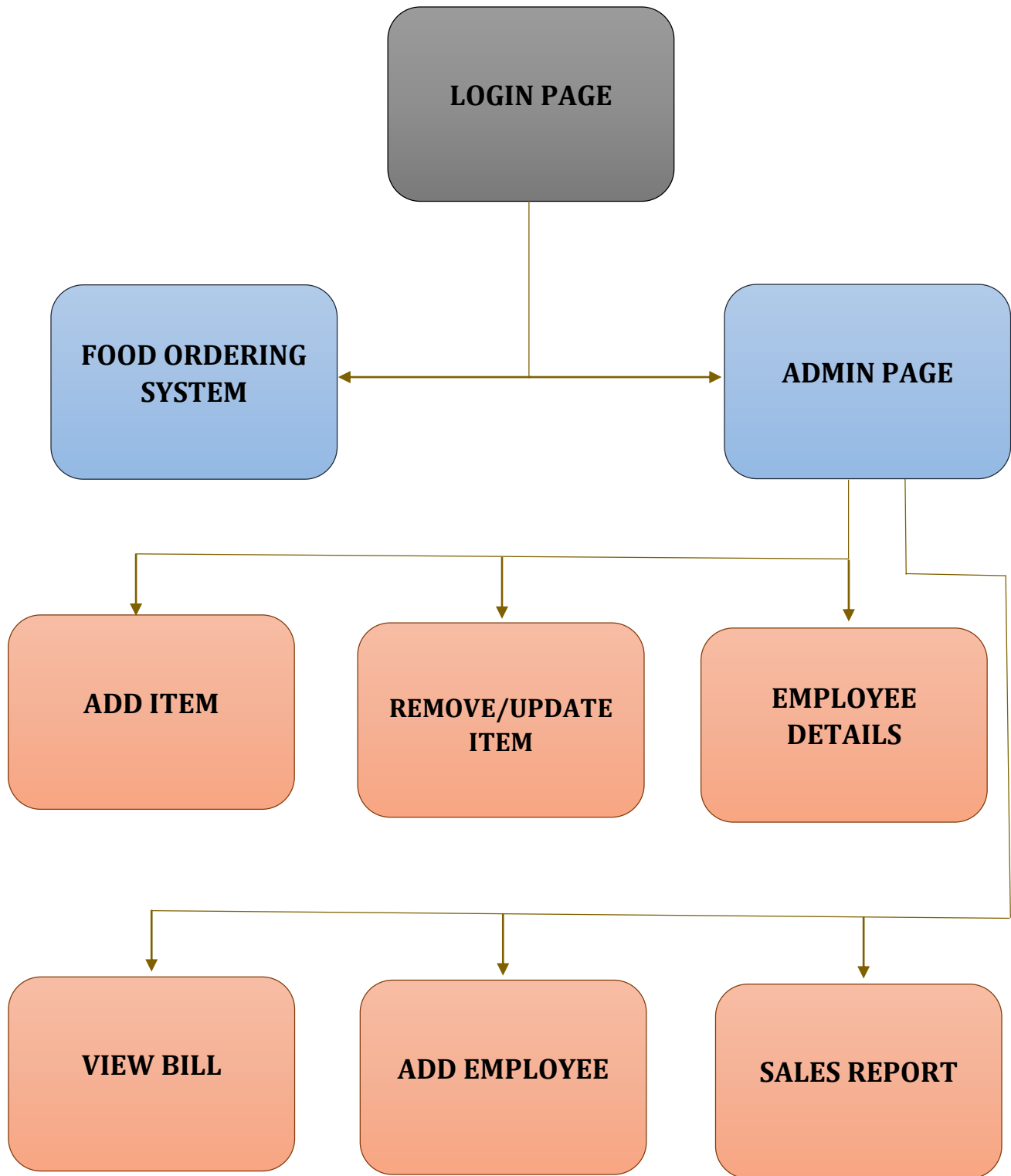
A **Food ordering system** is vital for the efficient operation of any food outlet. By digitalizing administrative processes and order management, it greatly reduces task completion time, enhancing overall operations. Key benefits include:

- ✚ **Streamlined Operations:** Orders are processed faster and more accurately, reducing wait times and boosting customer satisfaction.
- ✚ **Data Accuracy:** Digital storage prevents data loss and ensures up-to-date records, including order details and customer preferences.
- ✚ **Sales Tracking:** The system tracks sales daily, monthly, and yearly, aiding in trend analysis, inventory management, and business decisions.
- ✚ **Customer Interaction:** Features like order history enhance the customer experience by making it more engaging and personalized.
- ✚ **Efficient Management:** Simplifies tasks such as menu updates and sales report generation, allowing staff to focus on customer service.
- ✚ **Scalability:** Easily scales with business growth, handling increased order volumes without additional administrative burden.

MODULES

There are 9 modules in this software.

- ✚ **Login Module:** Allows the user to enter their ID and password, connecting them to the main page based on their role.
- ✚ **Food Ordering Module:** Enables the placement of customer orders efficiently.
- ✚ **Admin Module:** Enables the admin to manage items (add, remove, update), add new employees, view sales reports, and review previous orders and current items.
- ✚ **Add Item Module:** Facilitates adding new items along with all necessary details.
- ✚ **Remove/Update Item Module:** Lets you update or remove current items.
- ✚ **Employee Details Module:** Provides a comprehensive, searchable, and customizable view of employee information.
- ✚ **View Bill Module:** Shows previous orders and their detailed information.
- ✚ **Add Employee Module:** Allows the admin to add new employees and provide them with a password.
- ✚ **Sales Report Module:** Provides sales data based on day, month, or year.



SOFTWARES USED

The development of this application involved several key software tools:

- ✚ **PyCharm (2023.1):** A powerful Python Integrated Development Environment (IDE), PyCharm includes essential tools for Python developers, such as code analysis, a graphical debugger, and integrated unit testing. It also supports web development and data science integration, enhancing productivity and ease of development.
- ✚ **MySQL (SQL version 8.0):** Serving as the relational database management system, MySQL is known for its reliability and high performance. It stores and manages data efficiently, supporting complex queries and ensuring data integrity.
- ✚ **Python (3.11):** This version of Python brings significant performance improvements, with speed enhancements of 10% to 60%. It features better error messages, support for TOML, and task groups in asyncio, and new typing features, making it robust and efficient for development.
- ✚ **Qt Designer:** A visual tool for creating GUIs, Qt Designer allows developers to design interfaces by dragging and dropping widgets. It supports PyQt for seamless integration with Python code, enhancing productivity and ensuring user-friendly, visually appealing interfaces.

LIBRARIES USED

- ✚ **PyQt5:** PyQt5 is a Python binding for Qt, a set of C++ libraries and development tools providing platform-independent abstractions for graphical user interfaces (GUIs). Qt also includes tools for networking, threading, regular expressions, SQL databases, SVG, OpenGL, XML, and many other powerful features.
- ✚ **MySQL Connector:** MySQL Connectors provide connectivity to the MySQL server for client programs. These APIs offer low-level access to MySQL resources using either the classic MySQL protocol or X Protocol.
- ✚ **sys:** The sys module in Python provides various functions and variables that are used to manipulate different parts of the Python runtime environment. It allows for operations on the interpreter by providing access to variables and functions that interact closely with the interpreter.
- ✚ **Matplotlib:** Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It is widely used for its ability to produce high-quality plots, charts, and figures, making it essential for data analysis and presentation. Matplotlib integrates well with NumPy and Pandas, facilitating easy plotting of data from these libraries.

 **datetime:** The datetime module supplies classes for manipulating dates and times. It provides a simple yet efficient way to handle date and time arithmetic, supporting both simple and complex operations, such as calculating the difference between dates and formatting date strings.

 **traceback:** The traceback module provides a standard interface for extracting, formatting, and printing stack traces of a program. It is useful for error handling and debugging, as it helps developers understand the flow of execution that led to an error.

 **os:** The os module in Python provides functions to interact with the operating system, enabling tasks like file and directory management, process control, and handling environment variables. It allows manipulation of files, paths, and directories, execution of system commands, and management of processes, facilitating OS-level operations in Python.

 **Pyinstaller:** PyInstaller is a tool that converts Python scripts into standalone executables. It bundles the script with its dependencies, the Python interpreter, and required files into a single package. This allows the executable to run on systems without needing Python installed, simplifying distribution across different platforms.

Command: `pyinstaller --icon=icon.png --add-data "Images/*;images" --noconsole --onefile --name Food_Ordering_System Login.py`

SYSTEM REQUIREMENTS

OS Requirements:

1. Operating System:

- Windows 10 (64-bit) or later version.

2. Processor:

- Intel Core i3-6100 (3.7GHz) or equivalent AMD processor, dual-core or higher.
- Recommended: A quad-core processor for better performance, especially in multitasking or heavier workloads.

3. RAM:

- Minimum: 4GB RAM.
- Recommended: 8GB or more for better multitasking and performance during intensive tasks.

4. Storage:

- Minimum: 20GB of free disk space.
- Recommended: SSD (256GB or higher) for faster read/write speeds and better overall performance.

5. Graphics:

- Integrated Graphics: Sufficient for general usage and development.
- Dedicated GPU: Recommended for graphics-intensive tasks.

6. Display:

- Minimum resolution: 1366 x 768 for basic tasks.
- Recommended: Full HD (1920 x 1080) or higher for a better working environment.

Software Requirements:

1. Python 3.11:

- Python 3.11 or higher for development, running scripts, and using libraries.

2. MySQL 8.0:

- MySQL Server 8.0 or higher for database management and development.

DATABASE STRUCTURE

users
uid TINYINT
uname VARCHAR(20)
nname VARCHAR(6)
gender ENUM('M', 'F', 'O')
password VARCHAR(15)
uaddress VARCHAR(100)
email VARCHAR(50)
uphone VARCHAR(10)
Indexes

food_txn
order_id BIGINT
cust_name VARCHAR(50)
cust_phone VARCHAR(10)
item_name VARCHAR(100)
qty BIGINT
price VARCHAR(10)
total BIGINT
date_txn DATETIME

items
tid INT
category VARCHAR(15)
iname VARCHAR(50)
amt VARCHAR(4)
description VARCHAR(500)
image_path LONGBLOB
Indexes

DATABASE

1. Users Table:

Field	Type	Null	Key	Default	Extra
uid	tinyint	NO	PRI	NULL	
uname	varchar(20)	NO		NULL	
nname	varchar(6)	YES		NULL	
gender	enum('M','F','O')	YES		NULL	
password	varchar(15)	NO		NULL	
uaddress	varchar(100)	NO		NULL	
email	varchar(50)	NO		NULL	
uphone	varchar(10)	NO		NULL	

2. Items Table:

Field	Type	Null	Key	Default	Extra
tid	int	NO	PRI	NULL	
category	varchar(15)	YES		NULL	
iname	varchar(50)	YES		NULL	
amt	varchar(4)	YES		NULL	
description	varchar(500)	YES		NULL	
image_path	longblob	YES		NULL	

3. Food_txn Table:

Field	Type	Null	Key	Default	Extra
order_id	bigint	YES		NULL	
cust_name	varchar(50)	YES		NULL	
cust_phone	varchar(10)	YES		NULL	
item_name	varchar(100)	YES		NULL	
qty	bigint	YES		NULL	
price	varchar(10)	YES		NULL	
total	bigint	YES		NULL	
date_txn	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

PROGRAM CODE

1. LOGIN PAGE

```
import os
import traceback
import mysql.connector
from PyQt5 import QtWidgets, QtGui, QtCore
from PyQt5.QtWidgets import QApplication, QDialog
import sys

def resource_path(relative_path):
    try:
        if getattr(sys, 'frozen', False):
            base_path = sys._MEIPASS
        else:
            base_path = os.path.dirname(__file__)

        resource_full_path = os.path.join(base_path,
relative_path)

        return resource_full_path
    except Exception as e:
        print(f"Error in resource_path: {e}")
        return relative_path

class Login(QDialog):

    def __init__(self):
        super(Login, self).__init__()

        background_image =
resource_path('images/2.jpg')

        background_image_url =
background_image.replace("\\", "/")
```

```

        self.setStyleSheet(f'background-image:
url({background_image_url});')

        title_font = QtGui.QFont()
        title_font.setPointSize(21)
        title_font.setFamily("18th Century")
        title_font.setBold(True)

        headFont = QtGui.QFont()
        headFont.setPointSize(12)
        headFont.setFamily("18th Century")
        headFont.setBold(True)

        nextFont = QtGui.QFont()
        nextFont.setPointSize(12)
        nextFont.setFamily("18th Century")
        nextFont.setBold(True)

        tFont = QtGui.QFont()
        tFont.setPointSize(20)
        tFont.setFamily("Distant Galaxy")
        tFont.setBold(True)

        hFont = QtGui.QFont()
        hFont.setPointSize(8)
        hFont.setFamily("18th Century")
        hFont.setBold(True)

        self.setFixedWidth(350)
        self.setFixedHeight(300)

        self.setWindowTitle("LOGIN PAGE")

        self.head_le = QtWidgets.QLabel(self)
        self.head_le.setText("LOGIN")
        self.head_le.setFont(tFont)
        self.head_le.setGeometry(130, 30, 350, 50)
        self.head_le.setStyleSheet("background:
transparent; color: white;")

```

```

        self.uid_lbl = QtWidgets.QLabel(self)
        self.uid_lbl.setText("ID")
        self.uid_lbl.setGeometry(70, 110, 70, 30)
        self.uid_lbl.setFont(headFont)
        self.uid_lbl.setStyleSheet("background:
transparent; color: white;")

        self.uid_le = QtWidgets.QLineEdit(self)
        self.uid_le.setGeometry(180, 110, 120, 30)
        self.uid_le.setFont(headFont)
        self.uid_le.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.password_lbl = QtWidgets.QLabel(self)
        self.password_lbl.setText("PASSWORD")
        self.password_lbl.setGeometry(70, 150, 100, 30)
        self.password_lbl.setFont(headFont)
        self.password_lbl.setStyleSheet("background:
transparent; color: white;")

        self.password_le = QtWidgets.QLineEdit(self)
        self.password_le.setGeometry(180, 150, 120, 30)
        self.password_le.setFont(headFont)
        self.password_le.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.password_le.setEchoMode(QtWidgets.QLineEdit.Passwo
rd)

        self.show_password_checkbox =
QtWidgets.QCheckBox("SHOW PASSWORD", self)
        self.show_password_checkbox.setGeometry(180,
180, 120, 30)

        self.show_password_checkbox.stateChanged.connect(self.p
assword_visibility)
        self.show_password_checkbox.setFont(hFont)

```

```

self.show_password_checkbox.setStyleSheet("background:
transparent; color: white;")

self.login_btn = QtWidgets.QPushButton(self)
self.login_btn.setGeometry(120, 230, 100, 30)
self.login_btn.setText('LOGIN')
self.login_btn.setFont(nextFont)
self.login_btn.setStyleSheet("background:
rgba(0, 0, 0, 150); color: white; border: none;")
self.login_btn.clicked.connect(self.login)

try:
    self.db = mysql.connector.connect(
        host="127.0.0.1",
        user="root",
        passwd="sairam",
        database="project"
    )

    self.cursor = self.db.cursor()

    self.cursor.execute("""
        CREATE TABLE IF NOT EXISTS `users` (
            `uid` TINYINT NOT NULL,
            `uname` VARCHAR(20) NOT NULL,
            `nname` VARCHAR(6) DEFAULT NULL,
            `gender` ENUM('M', 'F', 'O')
DEFAULT NULL,
            `password` VARCHAR(15) NOT NULL,
            `uaddress` VARCHAR(100) NOT
NULL,
            `email` VARCHAR(50) NOT NULL,
            `uphone` VARCHAR(10) NOT NULL,
            PRIMARY KEY (`uid`)
        )
    """)

    self.cursor.execute("""
        CREATE TABLE IF NOT EXISTS `items` (

```

```

        `tid` INT NOT NULL,
        `category` VARCHAR(15) DEFAULT
NULL,
        `iname` VARCHAR(50) DEFAULT
NULL,
        `amt` VARCHAR(4) DEFAULT NULL,
        `description` VARCHAR(500)
DEFAULT NULL,
        `image_path` LONGBLOB,
        PRIMARY KEY (`tid`)
    )
    """)

```

```

        self.cursor.execute("""
            CREATE TABLE IF NOT EXISTS
`food_txn` (
                `order_id` BIGINT DEFAULT NULL,
                `cust_name` VARCHAR(50) DEFAULT
NULL,
                `cust_phone` VARCHAR(10)
DEFAULT NULL,
                `item_name` VARCHAR(100)
DEFAULT NULL,
                `qty` BIGINT DEFAULT NULL,
                `price` VARCHAR(10) DEFAULT
NULL,
                `total` BIGINT DEFAULT NULL,
                `date_txn` DATETIME DEFAULT
CURRENT_TIMESTAMP
            )
        """)

```

```

        self.cursor.execute(
            """INSERT IGNORE INTO `users` (uid,
password) VALUES (0, 'password1')"""
        )

```

```

    except Exception as e:
        traceback.print_exc()

```

```

        self.show_message("Sql      Error",      "Table
Creation Error")

    def login(self):
        self.uid = self.uid_le.text()
        password = self.password_le.text()

        query = "SELECT password FROM users WHERE uid =
%s"

        self.cursor.execute(query, (self.uid,))
        result = self.cursor.fetchone()

        try:
            if result:
                if password == result[0]:
                    if self.uid == "0":
                        self.admin()
                        self.destroy()
                    else:
                        self.secondPage()
                else:
                    self.show_message("Login      Error",
"Incorrect password")
            else:
                self.show_message("Login      Error",
"Username not found")
        except Exception as e:
            traceback.print_exc()
            self.show_message('Login Error', str(e))

        self.db.commit()

    def show_message(self, title, message):
        msg_box = QtWidgets.QMessageBox(self)
        msg_box.setWindowTitle(title)
        msg_box.setText(message)

        msg_box.setStyleSheet("background-color: white;
color: black;")

```

```

        msg_box.exec_()

    def secondPage(self):
        import prac4
        self.s = prac4.FoodApp()
        self.s.show()
        self.destroy()

    def password_visibility(self, state):
        if state == QtCore.Qt.Checked:

self.password_le.setEchoMode(QtWidgets.QLineEdit.Normal
)
            else:

self.password_le.setEchoMode(QtWidgets.QLineEdit.Passwo
rd)

    def admin(self):
        import admin_page
        self.s = admin_page.Admin()
        self.s.show()
        self.destroy()

if __name__ == '__main__':
    app = QApplication(sys.argv)
    l = Login()
    l.show()
    sys.exit(app.exec_())

```

2. FOOD ORDERING SYSTEM

```
import datetime
import os
import sys
import traceback
import mysql.connector
from PyQt5 import QtCore, QtWidgets, QtGui
from PyQt5.QtCore import QObject, pyqtSignal, Qt, QSize,
QRect
from PyQt5.QtGui import QFont
from PyQt5.QtWidgets import QMainWindow, QWidget, QVBoxLayout,
QLabel, \
    QPushButton, QListWidget, QListWidgetItem,
QScrollArea, QSizePolicy, QDialog, QHeaderView, QSpinBox

def resource_path(relative_path):
    try:
        if getattr(sys, 'frozen', False):
            base_path = sys._MEIPASS
        else:
            base_path = os.path.dirname(__file__)

        resource_full_path = os.path.join(base_path,
relative_path)

        return resource_full_path
    except Exception as e:
        print(f"Error in resource_path: {e}")
        return relative_path

class FoodItem:
    def __init__(self, name, price, image_path,
description):
        self.name = name
        self.price = price
        self.image_path = image_path
```

```

        self.description = description

class DataHandler(QObject):
    data_received = pyqtSignal(str, float, int)

class Confirm(QDialog):
    def __init__(self, food_app, parent=None):
        super().__init__(parent)
        self.food_app = food_app

        self.setWindowTitle("Confirmation")
        self.setFixedWidth(590)
        self.setFixedHeight(550)

        background_image =
resource_path('images/bg.jpg')

        background_image_url =
background_image.replace("\\", "/")

        self.setStyleSheet(f'background-image:
url({background_image_url});')
        cFont = QtGui.QFont()
        cFont.setPointSize(14)
        cFont.setFamily("Acme")
        cFont.setBold(True)

        oFont = QtGui.QFont()
        oFont.setPointSize(14)
        oFont.setFamily("Bahnschrift SemiBold
Condensed")
        oFont.setBold(True)

        self.conn = mysql.connector.connect(
            host="127.0.0.1",
            user="root",
            passwd="sairam",
            database="project"

```

```

    )

    self.cursor = self.conn.cursor()

    self.table2 = QtWidgets.QTableWidget(self)
    self.table2.setColumnCount(3)
    self.table2.setHorizontalHeaderLabels(["Items",
"Qty", "Rate"])
    self.table2.setGeometry(10, 140, 345, 400)
    self.table2.setFont(QFont("Minion Pro Med", 10))

self.table2.setEditTriggers(QtWidgets.QAbstractItemView
.NoEditTriggers)

self.table2.horizontalHeader().setSectionResizeMode(QHe
aderView.ResizeToContents)

self.table2.horizontalHeader().setSectionResizeMode(0,
QHeaderView.Stretch)

self.table2.horizontalHeader().setSectionResizeMode(1,
QHeaderView.ResizeToContents)

self.table2.horizontalHeader().setSectionResizeMode(2,
QHeaderView.ResizeToContents)

    self.table2.setStyleSheet("""
        QTableWidget {
            background-color: transparent;
        }
        QHeaderView::section {
            background-color: transparent;
/* Make headers transparent */
            color: black;                                /*
Change header text color to white */
            font-weight: bold;                            /*
Make header text bold */
        }
        QTableWidget::item {

```

```

        color: black;                                /*
Change item text color to white */
        font-weight: bold;                            /*
Make item text normal weight */
    }
    """)

self.table2.setAttribute(Qt.WA_TranslucentBackground)

    self.date_lbl = QtWidgets.QLabel(self)
    self.date_lbl.setText("DATE/TIME:")
    self.date_lbl.setGeometry(20, 15, 130, 30)
    self.date_lbl.setFont(cFont)
    self.date_lbl.setStyleSheet("background:
transparent; color: black;")

    self.date_le = QtWidgets.QLineEdit(self)
    self.date_le.setGeometry(145, 15, 340, 30)
    self.date_le.setFont(oFont)
    self.date_le.setReadOnly(True)
    self.date_le.setStyleSheet(
        "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: black; padding: 5px;")

    self.name = QtWidgets.QLabel(self)
    self.name.setGeometry(QtCore.QRect(20, 35, 170,
71))
    self.name.setText("NAME:")
    self.name.setFont(cFont)
    self.name.setStyleSheet("background:
transparent; color: black;")

    self.name_lbl = QtWidgets.QLineEdit(self)
    self.name_lbl.setGeometry(QtCore.QRect(145, 55,
340, 30))
    self.name_lbl.setFont(oFont)
    self.name_lbl.setStyleSheet(
        "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: black; padding: 5px;")

```

```

        self.name = self.name_lbl.text()

        self.num = QtWidgets.QLabel(self)
        self.num.setGeometry(QtCore.QRect(20, 75, 170,
71))
        self.num.setText("PHONE NO.:")
        self.num.setFont(cFont)
        self.num.setStyleSheet("background:
transparent; color: black;")

        self.num_lbl = QtWidgets.QLineEdit(self)
        self.num_lbl.setGeometry(QtCore.QRect(145, 95,
340, 30))
        self.num_lbl.setFont(oFont)
        self.num_lbl.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: black; padding: 5px;")

        self.output = QtWidgets.QLineEdit(self)
        self.output.setGeometry(360, 170, 220, 290)
        self.output.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: black; padding: 5px;")

        self.ord = QtWidgets.QLabel(self)
        self.ord.setGeometry(QtCore.QRect(370, 170,
210, 71))
        self.ord.setText("ORDER NO.:")
        self.ord.setFont(cFont)
        self.ord.setStyleSheet("background:
transparent; color: black;")

        self.ord_lbl = QtWidgets.QLineEdit(self)
        self.ord_lbl.setGeometry(QtCore.QRect(490, 190,
80, 30))
        self.ord_lbl.setFont(oFont)
        self.ord_lbl.setReadOnly(True)
        self.ord_lbl.setStyleSheet(

```

```

        "background: transparent; border: 1px solid
        rgba(255, 255, 255, 200); color: black; padding: 5px;")

        self.total_lbl = QtWidgets.QLabel(self)
        self.total_lbl.setText("TOTAL:")
        self.total_lbl.setGeometry(370, 235, 160, 30)
        self.total_lbl.setFont(cFont)
        self.total_lbl.setStyleSheet("background:
transparent; color: black;")

        self.total_le = QtWidgets.QLineEdit(self)
        self.total_le.setGeometry(445, 235, 125, 30)
        self.total_le.setFont(oFont)
        self.total_le.setReadOnly(True)
        self.total_le.setStyleSheet(
            "background: transparent; border: 1px solid
            rgba(255, 255, 255, 200); color: black; padding: 5px;")

        self.cgst_sgst = QtWidgets.QLabel(self)
        self.cgst_sgst.setGeometry(QtCore.QRect(370,
265, 170, 71))
        self.cgst_sgst.setText("CGST/SGST:")
        self.cgst_sgst.setFont(cFont)
        self.cgst_sgst.setStyleSheet("background:
transparent; color: black;")

        self.cgst_sgst_lbl = QtWidgets.QLineEdit(self)

        self.cgst_sgst_lbl.setGeometry(QtCore.QRect(490,      280,
80, 30))
        self.cgst_sgst_lbl.setFont(oFont)
        self.cgst_sgst_lbl.setReadOnly(True)
        self.cgst_sgst_lbl.setStyleSheet(
            "background: transparent; border: 1px solid
            rgba(255, 255, 255, 200); color: black; padding: 5px;")

        self.disc = QtWidgets.QLabel(self)
        self.disc.setGeometry(QtCore.QRect(370,      310,
170, 71))
        self.disc.setText("DISCOUNT:")

```

```

        self.disc.setFont(cFont)
        self.disc.setStyleSheet("background:
transparent; color: black;")

        self.disc_lbl = QtWidgets.QLineEdit(self)
        self.disc_lbl.setGeometry(QtCore.QRect(490,
325, 80, 30))
        self.disc_lbl.setFont(oFont)
        self.disc_lbl.setReadOnly(True)
        self.disc_lbl.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: black; padding: 5px;")
        self.update()

        self.total_bill_lbl = QtWidgets.QLabel(self)
        self.total_bill_lbl.setText("TOTAL AMT:")
        self.total_bill_lbl.setGeometry(370, 370, 160,
30)
        self.total_bill_lbl.setFont(cFont)
        self.total_bill_lbl.setStyleSheet("background:
transparent; color: black;")

        self.total_bill_le = QtWidgets.QLineEdit(self)
        self.total_bill_le.setGeometry(490, 370, 80, 30)
        self.total_bill_le.setFont(oFont)
        self.total_bill_le.setReadOnly(True)
        self.total_bill_le.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: black; padding: 5px;")

        self.itemsSold_lbl = QtWidgets.QLabel(self)
        self.itemsSold_lbl.setText("NO. OF ITEMS:")
        self.itemsSold_lbl.setGeometry(370, 410, 160,
30)
        self.itemsSold_lbl.setFont(cFont)
        self.itemsSold_lbl.setStyleSheet("background:
transparent; color: black;")

        self.itemsSold_le = QtWidgets.QLineEdit(self)
        self.itemsSold_le.setGeometry(515, 410, 55, 30)

```

```

        self.itemsSold_le.setFont(oFont)
        self.itemsSold_le.setReadOnly(True)
        self.itemsSold_le.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: black; padding: 5px;")

        self.conf = QtWidgets.QPushButton(self)
        self.conf.setGeometry(QtCore.QRect(400,      480,
100, 50))
        self.conf.setText("CONFIRM")
        self.conf.setFont(QFont("Arial Rounded MT Bold",
12))
        self.conf.clicked.connect(self.confirm_action)
        self.conf.setStyleSheet("color: black; border:
white;")

        self.back = QtWidgets.QPushButton(self)
        self.back.setText("BACK")
        self.back.setGeometry(510, 5, 70, 35)
        self.back.setFont(QFont("Arial Rounded MT Bold",
12))
        self.back.clicked.connect(self.go_back)
        self.back.setStyleSheet(" color: black; border:
white;")

        self.name = []
        self.qty = []
        self.pri = []
        self.total = []
        self.total_qty = 0
        self.item_total = 0

        id = "select max(order_id) from food_txn"
        self.cursor.execute(id)
        re = self.cursor.fetchone()
        self.id = int(re[0]) + 1

        self.current_datetime = datetime.datetime.now()
        self.formatted_datetime =
self.current_datetime.strftime('%Y-%m-%d %H:%M:%S')

```

```

def confirm_action(self):
    try:
        for row in range(self.table2.rowCount()):
            item_name = self.table2.item(row,
0).text()

            self.name.append(item_name)

            quantity = int(self.table2.item(row,
1).text())

            self.qty.append(int(quantity))

            price = float(self.table2.item(row,
2).text())

            self.pri.append(int(price))

            total_item_price = quantity * price

self.total.append(int(total_item_price))

            self.cust = self.name_lbl.text()
            self.phone = self.num_lbl.text()

            self.total_qty = sum(self.qty)

            if self.table2.rowCount() == 0:
                QtWidgets.QMessageBox.warning(self,
"ITEM ERROR", "NO ITEMS PLACED IN CART")

                if self.name_lbl.text() == "" or
len(self.num_lbl.text()) != 10:
                    QtWidgets.QMessageBox.warning(self,
"INFORMATION ERROR", "PLEASE FILL THE NECESSARY
INFORMATIONS")

                    if self.name_lbl.text() != "" and
len(self.num_lbl.text()) == 10:
                        for i in range(self.table2.rowCount()):
                            self.item_t = int(self.pri[i]) *
int(self.qty[i])

```

```

        self.item_total = self.item_t +
self.item_t * 0.05 - int(self.pri[i]) * 0.07
        sql_query = "INSERT INTO food_txn
VALUES ('{}', '{}', '{}', '{}', '{}', '{}', '{}',
'{}')".format(
            str(self.id), str(self.cust),
str(self.phone), str(self.name[i]), str(self.qty[i]),
            str(self.pri[i]),
str(self.item_total), self.formatted_datetime)
        self.cursor.execute(sql_query)

        self.conn.commit()

        QtWidgets.QMessageBox.warning(self,
'INSERTED', "ORDER PLACED")
        self.item_total = 0

        self.id += 1

        self.total_le.clear()
        self.cgst_sgst_lbl.clear()
        self.disc_lbl.clear()
        self.ord_lbl.clear()
        self.date_le.clear()
        self.total_bill_le.clear()
        self.itemsSold_le.clear()
        self.table2.setRowCount(0)
        self.name_lbl.clear()
        self.num_lbl.clear()

self.food_app.order_table.setRowCount(0)

self.food_app.total_price_label.setText("Total    Price:
$0.00")

        self.food_app.id += 1

        self.close()
        self.destroy()

```

```

        self.food_app.show()

    except Exception as a:
        traceback.print_exc()
        QtWidgets.QMessageBox.warning(self,
            "Error", str(a))
        return

    def go_back(self):
        self.close()
        self.food_app.show()

class FoodApp(QMainWindow):

    def __init__(self):
        super().__init__()

        self.setWindowTitle("FOOD APP")

        self.central_widget = QWidget()
        self.setCentralWidget(self.central_widget)

        background_image =
resource_path('images/bg.jpg')

        background_image_url =
background_image.replace("\\", "/")

        self.setStyleSheet(f'background-image:
url({background_image_url});')

        self.category_list =
QListWidget(self.central_widget)
        categories = ["Pizza", "Burger", "Pasta",
            "Momo", "Noodles", "Combos", "Donuts", "Cold Beverages",
            "Hot Beverages"]
        for cat in categories:
            item = QListWidgetItem(cat)
            self.category_list.addItem(item)

```

```

        item.setSizeHint(QSize(100, 50))
        self.category_list.setFont(QFont("Cracked
Johnnie", 12))
        self.category_list.setGeometry(QRect(20,      50,
230, 650))

        self.cat = QLabel(self)
        self.cat.setText("CATEGORY")
        self.cat.setFont(QFont("Distant Galaxy", 20))
        self.cat.setGeometry(70, 20, 150, 30)
        self.cat.setStyleSheet("background:
transparent; color: black;")

self.category_list.currentRowChanged.connect(self.show_
items)

        self.menu_items = []

        self.menu_area                                     =
QScrollArea(self.central_widget)
        self.menu_area.setWidgetResizable(True)

self.menu_area.setVerticalScrollBarPolicy(Qt.ScrollBarA
lwaysOn)
        self.menu_area.setGeometry(QRect(270,  50,  600,
650))

        self.menu_widget = QWidget()
        self.menu_layout = QVBoxLayout()
        self.menu_widget.setLayout(self.menu_layout)
        self.menu_area.setWidget(self.menu_widget)
        self.menu_area.verticalScrollBar().setValue(0)

        self.menu = QLabel(self)
        self.menu.setText("MENU")
        self.menu.setFont(QFont("Distant Galaxy", 20))
        self.menu.setGeometry(520, 20, 150, 30)
        self.menu.setStyleSheet("background:
transparent; color: black;")

```

```

        self.order_scroll_area = QScrollArea(self.central_widget)
        self.order_scroll_area.setWidgetResizable(True)

        self.order_scroll_area.setVerticalScrollBarPolicy(Qt.ScrollBarAlwaysOn)
        self.order_scroll_area.setGeometry(QRect(890, 50, 450, 650))
        self.order_table = QTableWidgetItem()
        self.order_table.setColumnCount(5)

        self.order_table.setHorizontalHeaderLabels(["Item", "Price", "Quantity", "Total", "Remove"])

        self.order_table.setEditTriggers(QTableWidgetItem.NoEditTriggers)

        self.order_table.horizontalHeader().setSectionResizeMode(
            QHeaderView.ResizeToContents)
        self.order_table.setFont(QFont("Minion Pro Med", 10))

        self.order_scroll_area.setWidget(self.order_table)

        self.order_table.setStyleSheet("""
            QTableWidgetItem {
                background-color:
transparent;
            }
            QHeaderView::section {
                background-color:
transparent; /* Make headers transparent */
                color: black;
/* Change header text color to white */
                font-weight: bold;
/* Make header text bold */
            }
            QTableWidgetItem::item {

```

```

                                color:                black;
/* Change item text color to white */
                                font-weight:           bold;
/* Make item text normal weight */
                                }
                                """)

self.order_table.setAttribute(Qt.WA_TranslucentBackgrou
nd)

self.order_table.horizontalHeader().setSectionResizemod
e(0, QHeaderView.Stretch)

self.order_table.horizontalHeader().setSectionResizemod
e(1, QHeaderView.ResizeToContents)

self.order_table.horizontalHeader().setSectionResizemod
e(2, QHeaderView.ResizeToContents)

self.order_table.horizontalHeader().setSectionResizemod
e(1, QHeaderView.ResizeToContents)

self.order_table.horizontalHeader().setSectionResizemod
e(2, QHeaderView.ResizeToContents)

        self.order = QLabel(self)
        self.order.setText("ORDER TABLE")
        self.order.setFont(QFont("Distant Galaxy", 20))
        self.order.setGeometry(1000, 20, 200, 30)
        self.order.setStyleSheet("background:
transparent; color: black;")

        self.total_price_label = QLabel("Total Price:
$0.00", self.central_widget)

self.total_price_label.setFont(QFont("Borealis", 11))
        self.total_price_label.setGeometry(QRect(900,
700, 500, 30))

```

```

self.total_price_label.setStyleSheet("background:
transparent; color: black;")

        self.proceed_button    =    QPushButton("PROCEED",
self.central_widget)
        self.proceed_button.setGeometry(QRect(1100,
702, 100, 30))

self.proceed_button.clicked.connect(self.proceed_to_con
firm)
        self.proceed_button.setFont(QFont("Arial
Rounded MT Bold", 10))
        self.proceed_button.setStyleSheet("        color:
black; border: white;")

        self.logout_button    =    QPushButton("LOGOUT↩",
self.central_widget)
        self.logout_button.setGeometry(QRect(1270, 10,
90, 30))

self.logout_button.clicked.connect(self.logout_action)
        self.logout_button.setFont(QFont("Arial Rounded
MT Bold", 10))
        self.logout_button.setStyleSheet("        color:
black; border: white;")

        self.total_price = 0.0
        self.update_total_price()

        self.data_handler = DataHandler()

self.data_handler.data_received.connect(self.receive_da
ta)

        self.name = []
        self.qty = []
        self.pri = []
        self.total = []
        self.id = 0

```

```

        self.total_qty = 0

        self.current_datetime = datetime.datetime.now()
        self.formatted_datetime =
self.current_datetime.strftime('%Y-%m-%d %H:%M:%S')

        self.conn = mysql.connector.connect(
            host="127.0.0.1",
            user="root",
            passwd="sairam",
            database="project"
        )

        self.cursor = self.conn.cursor()

        self.showMaximized()

    def logout_action(self):
        import Login
        self.s = Login.Login()
        self.s.show()
        self.destroy()

    def load_image(self, image_path):
        with open(image_path, 'rb') as file:
            image_data = file.read()
        self.pic = QtGui.QPixmap()
        self.pic.loadFromData(image_data)
        self.path.setPixmap(self.pic)

    def show_items(self, index):
        global image_path
        for i in
reversed(range(self.menu_layout.count())):

self.menu_layout.itemAt(i).widget().deleteLater()

        self.menu_items = []

        if index == 0:

```

```

        try:
            self.cursor.execute('SELECT iname, amt,
image_path, description FROM items WHERE category =
"pizza"')

            data = self.cursor.fetchall()

            for row in range(len(data)):
                name = data[row][0]
                price = data[row][1]
                image_path = data[row][2]
                description = data[row][3]

                food_item = FoodItem(name, price,
image_path, description)
                self.menu_items.append(food_item)

        except Exception as e:
            traceback.print_exc()
            QtWidgets.QMessageBox.warning(self,
"Error", str(e))
            return

    if index == 1:
        try:
            self.cursor.execute('SELECT iname, amt,
image_path, description FROM items WHERE category =
"burger"')

            data = self.cursor.fetchall()

            for row in range(len(data)):
                name = data[row][0]
                price = data[row][1]
                image_path = data[row][2]
                description = data[row][3]

                food_item = FoodItem(name, price,
image_path, description)
                self.menu_items.append(food_item)

        except Exception as e:

```

```

        traceback.print_exc()
        QtWidgets.QMessageBox.warning(self,
"Error", str(e))
        return

    if index == 2:
        try:
            self.cursor.execute('SELECT iname, amt,
image_path, description FROM items WHERE category =
"pasta"')

            data = self.cursor.fetchall()

            for row in range(len(data)):
                name = data[row][0]
                price = data[row][1]
                image_path = data[row][2]
                description = data[row][3]

                food_item = FoodItem(name, price,
image_path, description)
                self.menu_items.append(food_item)

        except Exception as e:
            traceback.print_exc()
            QtWidgets.QMessageBox.warning(self,
"Error", str(e))
            return

    if index == 3:
        try:
            self.cursor.execute('SELECT iname, amt,
image_path, description FROM items WHERE category =
"momo"')

            data = self.cursor.fetchall()

            for row in range(len(data)):
                name = data[row][0]
                price = data[row][1]
                image_path = data[row][2]
                description = data[row][3]

```

```

        food_item = FoodItem(name, price,
image_path, description)
        self.menu_items.append(food_item)

    except Exception as e:
        traceback.print_exc()
        QtWidgets.QMessageBox.warning(self,
"Error", str(e))
        return

    if index == 4:
        try:
            self.cursor.execute('SELECT iname, amt,
image_path, description FROM items WHERE category =
"noodles"')

            data = self.cursor.fetchall()

            for row in range(len(data)):
                name = data[row][0]
                price = data[row][1]
                image_path = data[row][2]
                description = data[row][3]

                food_item = FoodItem(name, price,
image_path, description)
                self.menu_items.append(food_item)

        except Exception as e:
            traceback.print_exc()
            QtWidgets.QMessageBox.warning(self,
"Error", str(e))
            return

    if index == 5:
        try:
            self.cursor.execute('SELECT iname, amt,
image_path, description FROM items WHERE category =
"combos"')

            data = self.cursor.fetchall()

```

```

        for row in range(len(data)):
            name = data[row][0]
            price = data[row][1]
            image_path = data[row][2]
            description = data[row][3]

            food_item = FoodItem(name, price,
image_path, description)
            self.menu_items.append(food_item)

        except Exception as e:
            traceback.print_exc()
            QtWidgets.QMessageBox.warning(self,
>Error", str(e))
            return

        if index == 6:
            try:
                self.cursor.execute('SELECT iname, amt,
image_path, description FROM items WHERE category =
"donuts"')

                data = self.cursor.fetchall()

                for row in range(len(data)):
                    name = data[row][0]
                    price = data[row][1]
                    image_path = data[row][2]
                    description = data[row][3]

                    food_item = FoodItem(name, price,
image_path, description)
                    self.menu_items.append(food_item)

                except Exception as e:
                    traceback.print_exc()
                    QtWidgets.QMessageBox.warning(self,
>Error", str(e))
                    return

```

```

        if index == 7:
            try:
                self.cursor.execute(
                    'SELECT  iname,  amt,  image_path,
description  FROM  items  WHERE  category  =  "cold
beverages"')
                data = self.cursor.fetchall()

                for row in range(len(data)):
                    name = data[row][0]
                    price = data[row][1]
                    image_path = data[row][2]
                    description = data[row][3]

                    food_item = FoodItem(name, price,
image_path, description)
                    self.menu_items.append(food_item)

            except Exception as e:
                traceback.print_exc()
                QtWidgets.QMessageBox.warning(self,
"Error", str(e))
                return

        if index == 8:
            try:
                self.cursor.execute(
                    'SELECT  iname,  amt,  image_path,
description  FROM  items  WHERE  category  =  "hot beverages"')
                data = self.cursor.fetchall()

                for row in range(len(data)):
                    name = data[row][0]
                    price = data[row][1]
                    image_path = data[row][2]
                    description = data[row][3]

                    food_item = FoodItem(name, price,
image_path, description)
                    self.menu_items.append(food_item)

```

```

        except Exception as e:
            traceback.print_exc()
            QtWidgets.QMessageBox.warning(self,
"Error", str(e))
            return

    try:
        for item in self.menu_items:
            item_widget = QWidget()
            item_layout = QVBoxLayout()
            item_widget.setLayout(item_layout)

            label      =      QLabel(f"{item.name}      -
₹{item.price:}")
            label.setFont(QFont("Candles", 15))
            label.setStyleSheet("background:
transparent; color: black;")

            item_layout.addWidget(label)

            if item.description:
                description_label      =
QLabel(item.description)

            description_label.setFont(QFont("Ink Free", 12))
            description_label.setWordWrap(True)

            description_label.setStyleSheet("background:
transparent; color: black; font-weight:bold")

            item_layout.addWidget(description_label)

            if item.image_path:
                pixmap = QtGui.QPixmap()

                pixmap.loadFromData(item.image_path)

                image_label = QLabel()
                image_label.setPixmap(pixmap)

```

```

image_label.setStyleSheet("background:      transparent;
color: black;")

        item_layout.addWidget(image_label)

        add_button = QPushButton("Add to Order")
        add_button.setFont(QFont("Arial Rounded
MT Bold", 10))
        add_button.clicked.connect(lambda _,
i=item: self.add_to_order(i))
        item_layout.addWidget(add_button)

item_widget.setSizePolicy(QSizePolicy.Expanding,
QSizePolicy.Fixed)
        self.menu_layout.addWidget(item_widget)

self.menu_area.verticalScrollBar().setValue(0)
        except Exception as e:
            traceback.print_exc()
            QtWidgets.QMessageBox.warning(self,
"Error", str(e))
            return

        def add_to_order(self, item):
            try:
                for row in
range(self.order_table.rowCount()):
                    existing_item_name =
self.order_table.item(row, 0).text()
                    if existing_item_name == item.name:
                        quantity_spinbox =
self.order_table.cellWidget(row, 2)

                        if quantity_spinbox:
                            new_quantity =
quantity_spinbox.value() + 1

```

```

quantity_spinbox.setValue(new_quantity)
                        self.update_row_total(row)
                        return

                        row_position = self.order_table.rowCount()
                        self.order_table.insertRow(row_position)

                        self.order_table.setItem(row_position,      0,
QTableWidgetItem(item.name))
                        self.order_table.setItem(row_position,      1,
QTableWidgetItem(str(item.price)))

                        quantity_spinbox = QSpinBox()
                        quantity_spinbox.setMinimum(1)
                        quantity_spinbox.setMaximum(100)
                        quantity_spinbox.setValue(1)
                        quantity_spinbox.setStyleSheet("          color:
black; border: white;")

self.order_table.setCellWidget(row_position,      2,
quantity_spinbox)

                        total_price_item =
QTableWidgetItem(str(item.price))
                        self.order_table.setItem(row_position,      3,
total_price_item)

                        remove_button = QPushButton("REMOVE")
                        remove_button.setFont(QFont("Arial   Rounded
MT Bold", 10))
                        remove_button.setStyleSheet("color:   black;
border: white;")

self.order_table.setCellWidget(row_position,      4,
remove_button)

                        remove_button.clicked.connect(lambda      _,
r=row_position: self.remove_row())

```

```

quantity_spinbox.valueChanged.connect(lambda _,
r=row_position: self.update_row_total(r))

self.order_table.horizontalHeader().setSectionResizeMod
e(0, QHeaderView.Stretch)

self.order_table.horizontalHeader().setSectionResizeMod
e(1, QHeaderView.ResizeToContents)

self.order_table.horizontalHeader().setSectionResizeMod
e(2, QHeaderView.ResizeToContents)

        self.update_total_price()

    except Exception as e:
        traceback.print_exc()
        QtWidgets.QMessageBox.warning(self,
"Error", str(e))

    def update_row_total(self, row):
        try:
            quantity =
float(self.order_table.cellWidget(row, 2).value())
            price = float(self.order_table.item(row,
1).text())
            total = quantity * price
            self.order_table.item(row,
3).setText(f"{total:.2f}")
            self.update_total_price()

        except Exception as e:
            traceback.print_exc()
            QtWidgets.QMessageBox.warning(self,
"Error", str(e))

    def remove_row(self):
        try:

```

```

        cur_row = self.order_table.currentRow()

        self.order_table.removeRow(cur_row)

        price_item = self.order_table.item(cur_row,
1)
        quantity_item =
self.order_table.cellWidget(cur_row, 2)
        self.update_total_price()

        if price_item and quantity_item:
            price = float(price_item.text())
            quantity = float(quantity_item.value())

            self.total_price -= price * quantity

    except Exception as e:
        traceback.print_exc()
        QtWidgets.QMessageBox.warning(self,
"Error", str(e))
        return

    def update_total_price(self):
        try:
            if self.order_table.rowCount() == 0:
                self.total_price_label.setText("Total
Price: ₹0")
                return
            total_price = 0
            for row in
range(self.order_table.rowCount()):
                total_price +=
float(self.order_table.item(row, 3).text())
            self.total_price_label.setText(f"Total
Price: ₹{total_price}")
        except Exception as e:
            traceback.print_exc()
            QtWidgets.QMessageBox.warning(self,
"Error", str(e))
            return

```

```

def receive_data(self, item_name, price, quantity):
    for row in range(self.order_table.rowCount()):
        if self.order_table.item(row, 0).text() ==
item_name:
            current_quantity =
int(self.order_table.item(row, 2).text())
            new_quantity = current_quantity +
quantity
            total_price = new_quantity * price
            self.order_table.setItem(row, 2,
QTableWidgetItem(str(new_quantity)))
            self.order_table.setItem(row, 3,
QTableWidgetItem(str(total_price)))
            self.update_total_price()
            return

        row_position = self.order_table.rowCount()
        self.order_table.insertRow(row_position)
        self.order_table.setItem(row_position, 0,
QTableWidgetItem(item_name))
        self.order_table.setItem(row_position, 1,
QTableWidgetItem(str(price)))
        self.order_table.setItem(row_position, 2,
QTableWidgetItem(str(quantity)))
        self.order_table.setItem(row_position, 3,
QTableWidgetItem(str(price * quantity)))
        self.update_total_price()

def proceed_to_confirm(self):
    confirm_dialog = Confirm(self)
    print("proceeding to confirm", confirm_dialog)

    row_count = self.order_table.rowCount()
    confirm_dialog.table2.setRowCount(row_count)

    for row in range(row_count):
        if int(self.order_table.cellWidget(row,
2).value()) > 0:

```

```

        item_name = self.order_table.item(row,
0).text()
        price
=
float(self.order_table.item(row, 1).text())
        quantity
=
int(self.order_table.cellWidget(row, 2).value())

        confirm_dialog.table2.setItem(row, 0,
QTableWidgetItem(item_name))
        confirm_dialog.table2.setItem(row, 1,
QTableWidgetItem(str(quantity)))
        confirm_dialog.table2.setItem(row, 2,
QTableWidgetItem(f"{price:.2f}"))

confirm_dialog.table2.horizontalHeader().setSectionResi
zeMode(0, QHeaderView.Stretch)

confirm_dialog.table2.horizontalHeader().setSectionResi
zeMode(1, QHeaderView.ResizeToContents)

confirm_dialog.table2.horizontalHeader().setSectionResi
zeMode(2, QHeaderView.ResizeToContents)

        self.name = []
        self.qty = []
        self.pri = []
        self.total = []

        try:
            for row in
range(confirm_dialog.table2.rowCount()):
                item_name
=
confirm_dialog.table2.item(row, 0).text()
                self.name.append(item_name)

                quantity
=
int(confirm_dialog.table2.item(row, 1).text())
                self.qty.append(quantity)

```

```

        price =
float(confirm_dialog.table2.item(row, 2).text())
        self.pri.append(price)

        total_item_price = quantity * price
        self.total.append(total_item_price)

food_txn"    id_query = "SELECT MAX(order_id) FROM

        self.cursor.execute(id_query)
        re = self.cursor.fetchone()
        self.id = int(re[0]) + 1

        self.random = confirm_dialog.id

        total_amount = sum(self.total)
        cgst_sgst = (total_amount * 0.05)
        discount = round(total_amount * 0.07)
        total_gst = cgst_sgst

        self.total_qty = sum(self.qty)

        total_amt = total_amount + total_gst -
discount

confirm_dialog.total_le.setText(f"{total_amount:.2f}")

confirm_dialog.cgst_sgst_lbl.setText(f"{cgst_sgst:.2f}"
)

confirm_dialog.disc_lbl.setText(f"{discount:.2f}")

confirm_dialog.ord_lbl.setText(str(self.random))

confirm_dialog.date_le.setText(self.formatted_datetime)

confirm_dialog.total_bill_le.setText(f"{total_amt:.2f}"
)

```

```
confirm_dialog.itemsSold_le.setText(str(self.total_qty)
)

        except Exception as e:
            traceback.print_exc()
            QtWidgets.QMessageBox.warning(self,
"Error", str(e))
            return
        self.close()
        confirm_dialog.exec_()
```

3. ADMIN PAGE

```
import os
import sys
from PyQt5 import QtCore, QtWidgets, QtGui
from PyQt5.QtWidgets import QMainWindow, QApplication

def resource_path(relative_path):
    try:
        if getattr(sys, 'frozen', False):
            base_path = sys._MEIPASS
        else:
            base_path = os.path.dirname(__file__)

        resource_full_path = os.path.join(base_path,
relative_path)

        return resource_full_path
    except Exception as e:
        print(f"Error in resource_path: {e}")
        return relative_path

class Admin(QMainWindow):
    def __init__(self):
        super().__init__()

        label_font = QtGui.QFont()
        label_font.setPointSize(11)
        label_font.setFamily('18th Century')
        label_font.setBold(True)

        nextFont = QtGui.QFont()
        nextFont.setPointSize(25)
        nextFont.setFamily("Distant Galaxy")
        nextFont.setBold(True)

        self.setFixedWidth(500)
        self.setFixedHeight(500)
```

```

        self.setWindowTitle("ADMIN PAGE")

        background_image = resource_path('images/3.jpg')

        background_image_url = background_image.replace("\\", "/")

        self.setStyleSheet(f'background-image: url({background_image_url});')

        self.add_item = QtWidgets.QPushButton(self)
        self.add_item.setText("ADD ITEM")
        self.add_item.setFont(label_font)
        self.add_item.setGeometry(60, 100, 180, 71)
        self.add_item.clicked.connect(self.add)
        self.add_item.setStyleSheet("background:
rgba(0, 0, 0, 150); color: white; border: black;")

        self.update = QtWidgets.QPushButton(self)
        self.update.setGeometry(300, 100, 180, 71)
        self.update.setText("UPDATE/REMOVE ITEM")
        self.update.setFont(label_font)
        self.update.clicked.connect(self.update_item)
        self.update.setStyleSheet("background: rgba(0,
0, 0, 150); color: white; border: black;")

        self.bill = QtWidgets.QPushButton(self)
        self.bill.setGeometry(300, 230, 180, 71)
        self.bill.setText("VIEW BILL")
        self.bill.setFont(label_font)
        self.bill.clicked.connect(self.bill_show)
        self.bill.setStyleSheet("background: rgba(0, 0,
0, 150); color: white; border: black;")

        self.add_member = QtWidgets.QPushButton(self)
        self.add_member.setGeometry(60, 360, 180, 71)
        self.add_member.setText("ADD EMPLOYEE")
        self.add_member.setFont(label_font)

```

```

        self.add_member.clicked.connect(self.sign_up)
        self.add_member.setStyleSheet("background:
rgba(0, 0, 0, 150); color: white; border: black;")

        self.report = QtWidgets.QPushButton(self)
        self.report.setGeometry(300, 360, 180, 71)
        self.report.setText("SALES REPORT")
        self.report.setFont(label_font)
        self.report.clicked.connect(self.s_report)
        self.report.setStyleSheet("background: rgba(0,
0, 0, 150); color: white; border: black;")

        self.id = QtWidgets.QPushButton(self)
        self.id.setGeometry(60, 230, 180, 71)
        self.id.setText("EMPLOYEE DETAILS")
        self.id.setFont(label_font)
        self.id.clicked.connect(self.know)
        self.id.setStyleSheet("background: rgba(0, 0, 0,
150); color: white; border: black;")

        self.back = QtWidgets.QPushButton(self)
        self.back.setGeometry(410, 10, 81, 41)
        self.back.setText("BACK")
        self.back.setFont(label_font)
        self.back.clicked.connect(self.login)
        self.back.setStyleSheet("background: rgba(0, 0,
0, 150); color: white; border: black;")

        self.admin = QtWidgets.QLabel(self)
        self.admin.setGeometry(QtCore.QRect(135, 20,
220, 61))
        self.admin.setFont(label_font)
        self.admin.setText("ADMIN PAGE")
        self.admin.setFont(nextFont)
        self.admin.setStyleSheet("background:
transparent; color: white;")

    def login(self):
        import Login
        self.s = Login.Login()

```

```
        self.s.show()
        self.destroy()

def add(self):
    import addItem
    self.r = addItem.add()
    self.r.show()
    self.destroy()

def know(self):
    import know_id
    self.r = know_id.id()
    self.r.show()
    self.destroy()

def bill_show(self):
    import show_bill
    self.r = show_bill.bill()
    self.r.show()
    self.destroy()

def sign_up(self):
    import signup
    self.s = signup.signup()
    self.s.show()
    self.destroy()

def s_report(self):
    import sales_report
    self.s = sales_report.sales()
    self.s.show()
    self.destroy()

def update_item(self):
    import update_remove_item
    self.s = update_remove_item.update()
    self.s.show()
    self.destroy()
```

4. ADD ITEM

```
import os
import sys
import traceback
from PyQt5 import QtWidgets, QtGui, QtCore
from PyQt5.QtWidgets import QMainWindow, QFileDialog
import mysql.connector

def resource_path(relative_path):
    try:
        if getattr(sys, 'frozen', False):
            base_path = sys._MEIPASS
        else:
            base_path = os.path.dirname(__file__)

        resource_full_path = os.path.join(base_path,
relative_path)

        return resource_full_path
    except Exception as e:
        print(f"Error in resource_path: {e}")
        return relative_path

class add(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setFixedWidth(800)
        self.setFixedHeight(420)
        self.setWindowTitle("ADD ITEM")

        background_image =
resource_path('images/1.jpg')

        background_image_url =
background_image.replace("\\", "/")
```

```

        self.setStyleSheet(f'background-image:
url({background_image_url});')

        label_font = QtGui.QFont()
        label_font.setPointSize(11)
        label_font.setFamily('18th Century')
        label_font.setBold(True)

        tFont = QtGui.QFont()
        tFont.setPointSize(24)
        tFont.setFamily("Distant Galaxy")
        tFont.setBold(True)

        try:
            self.db = mysql.connector.connect(
                host="127.0.0.1",
                user="root",
                passwd="sairam",
                database="project"
            )
            self.cursor = self.db.cursor()
            id_query = "select max(tid) from items"
            self.cursor.execute(id_query)
            re = self.cursor.fetchone()
            self.tid = int(re[0]) + 1
        except Exception as e:
            traceback.print_exc()
            QtWidgets.QMessageBox.warning(self, "Sql
Error", str(e))
            return

        self.add_items = QtWidgets.QLabel(self)
        self.add_items.setGeometry(350, 35, 220, 60)
        self.add_items.setText("ADD ITEM")
        self.add_items.setFont(tFont)
        self.add_items.setStyleSheet("background:
transparent; color: black;")

        self.back = QtWidgets.QPushButton(self)
        self.back.setGeometry(720, 15, 75, 23)

```

```

        self.back.setText("BACK")
        self.back.setFont(label_font)
        self.back.clicked.connect(self.admin)
        self.back.setStyleSheet("color: black; border:
black;")

        self.item_id = QtWidgets.QLabel(self)
        self.item_id.setGeometry(70, 115, 60, 13)
        self.item_id.setText("ITEM ID")
        self.item_id.setFont(label_font)
        self.item_id.setStyleSheet("background:
transparent; color: black;")

        self.id = QtWidgets.QLineEdit(self)
        self.id.setGeometry(190, 110, 261, 30)
        self.id.setReadOnly(True)
        self.id.setFont(label_font)
        self.id.setText(str(self.tid))
        self.id.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: black; padding: 5px;")

        self.category = QtWidgets.QLabel(self)
        self.category.setGeometry(70, 155, 100, 16)
        self.category.setText("CATEGORY")
        self.category.setFont(label_font)
        self.category.setStyleSheet("background:
transparent; color: black;")

        self.items = QtWidgets.QComboBox(self)
        self.items.setGeometry(190, 150, 261, 30)
        self.items.setFont(label_font)
        self.items.addItem(
            [" ", "Pizza", "Burger", "Pasta", "Momo",
"Noodles", "Combos", "Donuts", "Cold Beverages", "Hot
Beverages"])
        self.items.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: black; padding: 5px;")

```

```

        self.item_name = QtWidgets.QLabel(self)
        self.item_name.setGeometry(70, 195, 100, 16)
        self.item_name.setText("ITEM NAME")
        self.item_name.setFont(label_font)
        self.item_name.setStyleSheet("background:
transparent; color: black;")

        self.name = QtWidgets.QLineEdit(self)
        self.name.setGeometry(190, 190, 261, 30)
        self.name.setFont(label_font)
        self.name.setStyleSheet(
            "background: transparent; border: 1px solid
            rgba(255, 255, 255, 200); color: black; padding: 5px;")

        self.price = QtWidgets.QLabel(self)
        self.price.setGeometry(70, 245, 60, 13)
        self.price.setText("PRICE")
        self.price.setFont(label_font)
        self.price.setStyleSheet("background:
transparent; color: black;")

        self.pri = QtWidgets.QLineEdit(self)
        self.pri.setGeometry(190, 240, 261, 30)
        self.pri.setFont(label_font)
        self.pri.setStyleSheet(
            "background: transparent; border: 1px solid
            rgba(255, 255, 255, 200); color: black; padding: 5px;")

        self.description = QtWidgets.QLabel(self)
        self.description.setGeometry(70, 285, 110, 21)
        self.description.setText('DESCRIPTION')
        self.description.setFont(label_font)
        self.description.setStyleSheet("background:
transparent; color: black;")

        self.des = QtWidgets.QTextEdit(self)
        self.des.setGeometry(190, 280, 261, 55)
        self.des.setFont(label_font)
        self.des.setStyleSheet(

```

```

        "background: transparent; border: 1px solid
        rgba(255, 255, 255, 200); color: black; padding: 5px;")

        self.path = QtWidgets.QLabel(self)
        self.path.setGeometry(520, 120, 0, 0)
        self.path.setStyleSheet(
            "background: transparent; border: 1px solid
            rgba(255, 255, 255, 200); color: black; padding: 5px;")

        self.img_path = QtWidgets.QPushButton(self)
        self.img_path.setText("UPLOAD IMAGE")
        self.img_path.setGeometry(545, 300, 150, 30)
        self.img_path.setFont(label_font)
        self.img_path.setStyleSheet("color:          black;
border: black;")
        self.img_path.clicked.connect(self.image_path)

        self.add = QtWidgets.QPushButton(self)
        self.add.setGeometry(350, 360, 150, 30)
        self.add.setText("ADD ITEM")
        self.add.setFont(label_font)
        self.add.setStyleSheet("color:  black;  border:
black;")
        self.add.clicked.connect(self.insert)

    def image_path(self):
        try:
            self.file =
            QFileDialog.getOpenFileName(self, "Open File", "", "JPG
            Files (*.jpg);;PNG Files (*.png)")
            if self.file[0]:
                self.photo =
                QtGui.QPixmap(self.file[0])
                self.resize = self.photo.scaled(220,
                250, QtCore.Qt.KeepAspectRatio)
                self.path.setPixmap(self.resize)

                self.path.setGeometry(520,          120,
                self.resize.width(), self.resize.height())

```

```

        file = open(self.file[0], "rb")
        self.filedata = file.read()

    except Exception as e:
        traceback.print_exc()
        QtWidgets.QMessageBox.warning(self, "Sql
Error", str(e))
        return

    def insert(self):
        try:
            self.cat = self.items.currentText().lower()
            self.i_name = self.name.text()
            self.amt = self.pri.text()
            self.desc = self.des.toPlainText()

            if self.cat == " " or self.i_name == "" or
self.amt == "" or self.desc == "" or self.file[0] == "":
                QtWidgets.QMessageBox.warning(self,
"ERROR", 'PLEASE ENTER ALL THE ABOVE INFORMATION')
                return
            else:
                query = "INSERT INTO items (tid,
category, iname, amt, description, image_path) VALUES
(%s, %s, %s, %s, %s, %s)"
                values = (self.tid, self.cat,
self.i_name, self.amt, self.desc, self.filedata)

                self.cursor.execute(query, values)
                self.db.commit()

                QtWidgets.QMessageBox.warning(self,
"SUCCESSFUL", f'ITEM ADDED')

                self.tid += 1
                self.id.setText(str(self.tid))
                self.items.setCurrentText(" ")
                self.name.clear()
                self.pri.clear()
                self.des.clear()

```

```
        self.path.clear()

self.path.setStyleSheet('background:transparent')

        except Exception as e:
            traceback.print_exc()
            QtWidgets.QMessageBox.warning(self, "Sql
Error", str(e))
            return

def admin(self):
    import admin_page
    self.s = admin_page.Admin()
    self.s.show()
    self.destroy()
```

5. REMOVE/UPDATE ITEM

```
import os
import sys
import traceback
from PyQt5 import QtWidgets, QtGui, QtCore
from PyQt5.QtCore import Qt
from PyQt5.QtWidgets import QMainWindow, QFileDialog,
QTableWidget, QTableWidgetItem, QHeaderView
import mysql.connector

def resource_path(relative_path):
    try:
        if getattr(sys, 'frozen', False):
            base_path = sys._MEIPASS
        else:
            base_path = os.path.dirname(__file__)

        resource_full_path = os.path.join(base_path,
relative_path)

        return resource_full_path
    except Exception as e:
        print(f"Error in resource_path: {e}")
        return relative_path

class update(QMainWindow):
    def __init__(self):
        super().__init__()

        self.filedata = None
        self.random = 0
        self.setFixedWidth(1200)
        self.setFixedHeight(420)

        self.setWindowTitle("UPDATE/REMOVE ITEM")
```

```

        background_image                                     =
        resource_path('images/6.jpg')

        background_image_url                                 =
        background_image.replace("\\", "/")

        self.setStyleSheet(f'background-image:
        url({background_image_url});')

        label_font = QtGui.QFont()
        label_font.setPointSize(11)
        label_font.setFamily('18th Century')
        label_font.setBold(True)

        tFont = QtGui.QFont()
        tFont.setPointSize(24)
        tFont.setFamily("Distant Galaxy")
        tFont.setBold(True)

        self.db = mysql.connector.connect(
            host="127.0.0.1",
            user="root",
            passwd="sairam",
            database="project"
        )
        self.cursor = self.db.cursor()

        self.ur_items = QtWidgets.QLabel(self)
        self.ur_items.setGeometry(450, 0, 400, 60)
        self.ur_items.setText("REMOVE/UPDATE ITEM")
        self.ur_items.setFont(tFont)
        self.ur_items.setStyleSheet("background:
transparent; color: white;")

        self.back = QtWidgets.QPushButton(self)
        self.back.setGeometry(1120, 10, 75, 23)
        self.back.setText("BACK")
        self.back.setFont(label_font)
        self.back.clicked.connect(self.admin)

```

```

        self.back.setStyleSheet("background: rgba(0, 0,
0, 150); color: white; border: none;")

        self.item_id = QtWidgets.QLabel(self)
        self.item_id.setGeometry(70, 115, 60, 13)
        self.item_id.setText("ITEM ID")
        self.item_id.setFont(label_font)
        self.item_id.setStyleSheet("background:
transparent; color: white;")

        self.id = QtWidgets.QComboBox(self)
        self.id.setGeometry(190, 110, 261, 30)
        self.id.setFont(label_font)
        self.id.setEditable(True)

self.id.setInsertPolicy(QtWidgets.QComboBox.NoInsert)

self.id.completer().setCompletionMode(QtWidgets.QComple
ter.PopupCompletion)
        self.id.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.category = QtWidgets.QLabel(self)
        self.category.setGeometry(70, 155, 100, 16)
        self.category.setText("CATEGORY")
        self.category.setFont(label_font)
        self.category.setStyleSheet("background:
transparent; color: white;")

        self.items = QtWidgets.QLineEdit(self)
        self.items.setGeometry(190, 150, 261, 30)
        self.items.setFont(label_font)
        self.items.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.item_name = QtWidgets.QLabel(self)
        self.item_name.setGeometry(70, 195, 100, 16)
        self.item_name.setText("ITEM NAME")

```

```

        self.item_name.setFont(label_font)
        self.item_name.setStyleSheet("background:
transparent; color: white;")

        self.name = QtWidgets.QLineEdit(self)
        self.name.setGeometry(190, 190, 261, 30)
        self.name.setFont(label_font)
        self.name.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.price = QtWidgets.QLabel(self)
        self.price.setGeometry(70, 245, 60, 13)
        self.price.setText("PRICE")
        self.price.setFont(label_font)
        self.price.setStyleSheet("background:
transparent; color: white;")

        self.pri = QtWidgets.QLineEdit(self)
        self.pri.setGeometry(190, 240, 261, 30)
        self.pri.setFont(label_font)
        self.pri.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.description = QtWidgets.QLabel(self)
        self.description.setGeometry(70, 285, 110, 21)
        self.description.setText('DESCRIPTION')
        self.description.setFont(label_font)
        self.description.setStyleSheet("background:
transparent; color: white;")

        self.des = QtWidgets.QTextEdit(self)
        self.des.setGeometry(190, 280, 261, 55)
        self.des.setFont(label_font)
        self.des.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.path = QtWidgets.QLabel(self)

```

```

self.path.setGeometry(545, 120, 0, 0)
self.path.setStyleSheet(
    "background: transparent; border: 1px solid
    rgba(255, 255, 255, 200); color: white; padding: 5px;")

self.img_path = QtWidgets.QPushButton(self)
self.img_path.setText("UPLOAD IMAGE")
self.img_path.setGeometry(500, 300, 150, 30)
self.img_path.setFont(label_font)
self.img_path.clicked.connect(self.image_path)
self.img_path.setStyleSheet("background:
    rgba(0, 0, 0, 150); color: white; border: none;")

self.remove = QtWidgets.QPushButton(self)
self.remove.setGeometry(420, 380, 150, 30)
self.remove.setText("REMOVE ITEM")
self.remove.setFont(label_font)
self.remove.clicked.connect(self.delete_item)
self.remove.setStyleSheet("background:
    rgba(0, 0, 150); color: white; border: none;")

self.update = QtWidgets.QPushButton(self)
self.update.setGeometry(230, 380, 150, 30)
self.update.setText("UPDATE ITEM")
self.update.setFont(label_font)
self.update.clicked.connect(self.update_item)
self.update.setStyleSheet("background:
    rgba(0, 0, 150); color: white; border: none;font-weight:
    bold;")

self.num = "SELECT tid FROM items WHERE tid >0"
self.cursor.execute(self.num)

self.tid = [None]

self.id.addItem([""])
for i in self.cursor:
    self.tid = list(i)
    self.id.addItem(str(self.tid[0]))

```

```

self.id.currentTextChanged.connect(self.get_info)

        self.see_id = QtWidgets.QLineEdit(self)
        self.see_id.setGeometry(700, 55, 480, 30)
        self.see_id.setFont(label_font)
        self.see_id.setPlaceholderText('SEARCH      ITEM
NAME Q')
        self.see_id.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; ")

        self.id_table = QtWidgets.QTableWidget(self)
        self.id_table.setColumnCount(4)
        self.id_table.setGeometry(700, 90, 480, 320)
        self.id_table.setHorizontalHeaderLabels(["ID",
"CATEGORY", "ITEM", "PRICE"])

self.id_table.setEditTriggers(QtWidgets.QAbstractItemVi
ew.NoEditTriggers)

self.id_table.horizontalHeader().setSectionResizeMode(Q
HeaderView.ResizeToContents)

self.id_table.horizontalHeader().setSectionResizeMode(2
, QHeaderView.Stretch)

self.id_table.horizontalHeader().setSectionResizeMode(1
, QHeaderView.ResizeToContents)

self.id_table.horizontalHeader().setSectionResizeMode(0
, QHeaderView.ResizeToContents)

self.id_table.horizontalHeader().setSectionResizeMode(3
, QHeaderView.ResizeToContents)

        self.id_table.setStyleSheet("""
            QTableWidget {
                background-color: transparent;
            }

```

```

        QHeaderView::section {
            background-color: transparent;
/* Make headers transparent */
            color: white;                                /*
Change header text color to white */
            font-weight: bold;                            /*
Make header text bold */
        }
        QTableWidgetItem::item {
            color: white;                                /*
Change item text color to white */
            font-weight: bold;                            /*
Make item text normal weight */
        }
        QTableWidgetItem[data-
role="item"] {
            font-weight: bold;                            /*
Make item name text bold if necessary */
        }
        """)

self.id_table.setAttribute(Qt.WA_TranslucentBackground)

        query = "SELECT tid,category,iname,amt      FROM
items WHERE tid>0 ORDER BY category DESC"
        self.cursor.execute(query)

        rows = self.cursor.fetchall()

        self.id_table.setRowCount(len(rows))
        self.id_table.setColumnCount(len(rows[0]))

        for i, row in enumerate(rows):
            for j, col in enumerate(row):
                self.id_table.setItem(i,                                j,
QTableWidgetItem(str(col)))

        self.see_id.textEdited.connect(self.show_id)

```

```

def show_id(self):
    try:
        self.cursor.execute(f"SELECT * FROM items
WHERE iname like '%{self.see_id.text()}%' AND tid > 0")

        rows = self.cursor.fetchall()

        self.id_table.setRowCount(len(rows))
        for i, row in enumerate(rows):
            for j, col in enumerate(row):
                self.id_table.setItem(i, j,
QTableWidgetItem(str(col)))
    except Exception as e:
        traceback.print_exc()
        QtWidgets.QMessageBox.warning(self, "Table
Error", str(e))
        return

def image_path(self):
    try:
        self.file = QFileDialog()
        self.file =
QFileDialog.getOpenFileName(self, "Open File", "", "JPG
Files (*.jpg);;PNG Files (*.png)")
        if self.file[0]:
            self.photo =
QtGui.QPixmap(self.file[0])
            self.resize = self.photo.scaled(220,
210, QtCore.Qt.KeepAspectRatio)
            self.path.setPixmap(self.resize)

            self.path.setGeometry(470, 120,
self.resize.width(), self.resize.height())

            file = open(self.file[0], "rb")
            self.filedata = file.read()
            self.random += 1

    except Exception as e:
        traceback.print_exc()

```

```

        QtWidgets.QMessageBox.warning(self,
        "Inserting Error", str(e))
        return

    def get_info(self):
        self.tid = self.id.currentText()

        query = "SELECT
iname,category,amt,description,image_path FROM items
WHERE tid = %s"
        self.cursor.execute(query, (self.tid,))
        result = self.cursor.fetchone()

        try:
            self.name.setText(result[0])
            self.items.setText(result[1].capitalize())
            self.pri.setText(result[2])
            self.des.setText(result[3])

            pixmap = QtGui.QPixmap()
            pixmap.loadFromData(result[4])
            self.path.setPixmap(pixmap)
            self.path.setGeometry(480, 120,
pixmap.width(), pixmap.height())

        except Exception as e:
            traceback.print_exc()
            QtWidgets.QMessageBox.warning(self,
            "Uploading Error", str(e))
            return

    def update_item(self):
        id = self.id.currentText()
        name = self.name.text()
        amt = self.pri.text()
        desc = self.des.toPlainText()

        if self.random != 0:
            query = 'UPDATE items SET iname = %s, amt =
%s, description = %s, image_path = %s WHERE tid = %s'

```

```

        self.cursor.execute(query, (name, amt, desc,
self.filedata, id))
        self.db.commit()
    else:
        query = 'UPDATE items SET iname = %s, amt =
%s, description = %s WHERE tid = %s'
        self.cursor.execute(query, (name, amt, desc,
id))
        self.db.commit()

    QtWidgets.QMessageBox.warning(self,      "UPDAtE",
"ITEM UPDATED")

    self.id.setCurrentText(" ")
    self.items.clear()
    self.name.clear()
    self.pri.clear()
    self.des.clear()
    self.path.clear()
    self.see_id.clear()

    self.id_table.clear()
    self.id_table.setHorizontalHeaderLabels(["ID",
"CATEGORY", "ITEM", "PRICE"])

    query = "SELECT tid,category,iname,amt      FROM
items WHERE tid>0 ORDER BY category DESC"
    self.cursor.execute(query)

    rows = self.cursor.fetchall()

    self.id_table.setRowCount(len(rows))
    self.id_table.setColumnCount(len(rows[0]))

    for i, row in enumerate(rows):
        for j, col in enumerate(row):
            self.id_table.setItem(i,                j,
QTableWidgetItem(str(col)))

    def delete_item(self):

```

```

        try:
            if self.id.currentText() == "":
                QtWidgets.QMessageBox.warning(self,
"ERROR", "ENTER ITEM ID")
                return
            else:
                remove = "DELETE FROM items WHERE tid =
%s"
                self.cursor.execute(remove,
(self.tid,))
                self.db.commit()

                index = self.id.currentIndex()
                self.id.removeItem(index)
                self.id.setCurrentText(" ")

                QtWidgets.QMessageBox.warning(self,
"REMOVED", "ITEM REMOVED")

                self.items.clear()
                self.name.clear()
                self.pri.clear()
                self.des.clear()
                self.path.clear()
                self.see_id.clear()

                self.id_table.clear()

self.id_table.setHorizontalHeaderLabels(["ID",
"CATEGORY", "ITEM", "PRICE"])

                query = "SELECT tid,category,iname,amt
FROM items WHERE tid>0 ORDER BY category DESC"
                self.cursor.execute(query)

                rows = self.cursor.fetchall()

                self.id_table.setRowCount(len(rows))

self.id_table.setColumnCount(len(rows[0]))

```

```

        for i, row in enumerate(rows):
            for j, col in enumerate(row):
                self.id_table.setItem(i, j,
QTableWidgetItem(str(col)))

    except Exception as e:
        traceback.print_exc()
        QtWidgets.QMessageBox.warning(self, "Sql
Error", str(e))
        return

def admin(self):
    import admin_page
    self.s = admin_page.Admin()
    self.s.show()
    self.destroy()

```

6. EMPLOYEE DETAILS

```
import os
import sys
import traceback
import mysql.connector
from PyQt5 import QtWidgets, QtGui
from PyQt5.QtCore import Qt
from PyQt5.QtWidgets import QMainWindow,
QTableWidgetItem, QHeaderView

def resource_path(relative_path):
    try:
        if getattr(sys, 'frozen', False):
            base_path = sys._MEIPASS
        else:
            base_path = os.path.dirname(__file__)

        resource_full_path = os.path.join(base_path,
relative_path)

        return resource_full_path
    except Exception as e:
        print(f"Error in resource_path: {e}")
        return relative_path

class id(QMainWindow):
    def __init__(self):
        super().__init__()

        self.filedata = None
        self.setFixedWidth(980)
        self.setFixedHeight(420)

        self.setWindowTitle("UPDATE/REMOVE ITEM")
```

```

        background_image                                     =
resource_path('images/5.jpg')

        background_image_url                               =
background_image.replace("\\", "/")

        self.setStyleSheet(f'background-image:
url({background_image_url});')

        label_font = QtGui.QFont()
        label_font.setPointSize(11)
        label_font.setFamily('18th Century')
        label_font.setBold(True)

        tFont = QtGui.QFont()
        tFont.setPointSize(24)
        tFont.setFamily("Distant Galaxy")
        tFont.setBold(True)

        self.db = mysql.connector.connect(
            host="127.0.0.1",
            user="root",
            passwd="sairam",
            database="project"
        )
        self.cursor = self.db.cursor()

        self.lbl = QtWidgets.QLabel(self)
        self.lbl.setGeometry(300, 0, 400, 60)
        self.lbl.setText("EMPLOYEE DETAILS")
        self.lbl.setFont(tFont)
        self.lbl.setStyleSheet("background:
transparent; color: white;")

        self.back = QtWidgets.QPushButton(self)
        self.back.setGeometry(900, 10, 75, 23)
        self.back.setText("BACK")
        self.back.setFont(label_font)
        self.back.clicked.connect(self.admin)

```

```

        self.back.setStyleSheet("background: rgba(0, 0,
0, 150); color: white; border: none;")

        self.emp_id = QtWidgets.QLabel(self)
        self.emp_id.setGeometry(70, 115, 110, 13)
        self.emp_id.setText("EMPLOYEE ID")
        self.emp_id.setFont(label_font)
        self.emp_id.setStyleSheet("background:
transparent; color: white;")

        self.id = QtWidgets.QComboBox(self)
        self.id.setGeometry(190, 110, 261, 30)
        self.id.setFont(label_font)
        self.id.setEditable(True)

self.id.setInsertPolicy(QtWidgets.QComboBox.NoInsert)

self.id.completer().setCompletionMode(QtWidgets.QComple
ter.PopupCompletion)
        self.id.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.uname = QtWidgets.QLabel(self)
        self.uname.setGeometry(70, 155, 100, 16)
        self.uname.setText("NAME")
        self.uname.setFont(label_font)
        self.uname.setStyleSheet("background:
transparent; color: white;")

        self.name = QtWidgets.QLineEdit(self)
        self.name.setGeometry(190, 150, 261, 30)
        self.name.setFont(label_font)
        self.name.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.address = QtWidgets.QLabel(self)
        self.address.setGeometry(70, 195, 100, 16)
        self.address.setText("ADDRESS")

```

```

        self.address.setFont(label_font)
        self.address.setStyleSheet("background:
transparent; color: white;")

        self.add = QtWidgets.QLineEdit(self)
        self.add.setGeometry(190, 190, 261, 30)
        self.add.setFont(label_font)
        self.add.setStyleSheet(
            "background: transparent; border: 1px solid
            rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.email = QtWidgets.QLabel(self)
        self.email.setGeometry(70, 245, 60, 13)
        self.email.setText("EMAIL")
        self.email.setFont(label_font)
        self.email.setStyleSheet("background:
transparent; color: white;")

        self.uemail = QtWidgets.QLineEdit(self)
        self.uemail.setGeometry(190, 240, 261, 30)
        self.uemail.setFont(label_font)
        self.uemail.setStyleSheet(
            "background: transparent; border: 1px solid
            rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.password = QtWidgets.QLabel(self)
        self.password.setGeometry(70, 285, 110, 21)
        self.password.setText('PASSWORD')
        self.password.setFont(label_font)
        self.password.setStyleSheet("background:
transparent; color: white;")

        self.upass = QtWidgets.QLineEdit(self)
        self.upass.setGeometry(190, 280, 261, 55)
        self.upass.setFont(label_font)
        self.upass.setStyleSheet(
            "background: transparent; border: 1px solid
            rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.remove = QtWidgets.QPushButton(self)

```

```

        self.remove.setGeometry(300, 380, 150, 30)
        self.remove.setText("REMOVE ITEM")
        self.remove.setFont(label_font)
        self.remove.clicked.connect(self.delete_emp)
        self.remove.setStyleSheet("background: rgba(0,
0, 0, 150); color: white; border: none;")

        self.update = QtWidgets.QPushButton(self)
        self.update.setGeometry(100, 380, 150, 30)
        self.update.setText("UPDATE ITEM")
        self.update.setFont(label_font)
        self.update.clicked.connect(self.update_emp)
        self.update.setStyleSheet("background: rgba(0,
0, 0, 150); color: white; border: none;font-weight:
bold;")

        self.num = "SELECT uid FROM users WHERE uid >0"
        self.cursor.execute(self.num)

        self.tid = [None]

        self.id.addItem("")
        for i in self.cursor:
            self.tid = list(i)
            self.id.addItem(str(self.tid[0]))

        self.id.currentTextChanged.connect(self.get_info)

        self.see_id = QtWidgets.QLineEdit(self)
        self.see_id.setGeometry(480, 55, 480, 30)
        self.see_id.setFont(label_font)
        self.see_id.setPlaceholderText('SEARCH EMPLOYEE
NAME Q')
        self.see_id.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; ")
        try:
            self.id_table =
QtWidgets.QTableWidget(self)

```

```

        self.id_table.setColumnCount(5)
        self.id_table.setGeometry(480, 90, 480, 320)

self.id_table.setHorizontalHeaderLabels(["UID", "NAME",
"ADDRESS", "EMAIL", "PASSWORD"])

self.id_table.setEditTriggers(QtWidgets.QAbstractItemVi
ew.NoEditTriggers)

self.id_table.horizontalHeader().setSectionResizeMode(Q
HeaderView.ResizeToContents)

self.id_table.horizontalHeader().setSectionResizeMode(2
, QHeaderView.Stretch)

self.id_table.horizontalHeader().setSectionResizeMode(1
, QHeaderView.ResizeToContents)

self.id_table.horizontalHeader().setSectionResizeMode(0
, QHeaderView.ResizeToContents)

self.id_table.horizontalHeader().setSectionResizeMode(3
, QHeaderView.ResizeToContents)

self.id_table.horizontalHeader().setSectionResizeMode(4
, QHeaderView.ResizeToContents)

        self.id_table.setStyleSheet("""
                                QTableWidget {
                                    background-color:
transparent;
                                }
                                QHeaderView::section {
                                    background-color:
transparent; /* Make headers transparent */
                                    color:                                white;
/* Change header text color to white */
                                    font-weight:                        bold;
/* Make header text bold */
                                }

```

```

        QTableWidgetItem::item {
            color: white;
/* Change item text color to white */
            font-weight: bold;
/* Make item text normal weight */
        }
    """)

self.id_table.setAttribute(Qt.WA_TranslucentBackground)

        query = "SELECT
uid,uname,uaddress,email,password FROM users WHERE
uid>0"

        self.cursor.execute(query)

        rows = self.cursor.fetchall()

        self.id_table.setRowCount(len(rows))
        self.id_table.setColumnCount(len(rows[0]))

        for i, row in enumerate(rows):
            for j, col in enumerate(row):
                self.id_table.setItem(i, j,
QTableWidgetItem(str(col)))

self.see_id.textEdited.connect(self.show_id)
except Exception as e:
    print(e)

def admin(self):
    import admin_page
    self.s = admin_page.Admin()
    self.s.show()
    self.destroy()

def show_id(self):
    try:
        self.cursor.execute(

```

```

        f"SELECT
uid,uname,uaddress,email,password FROM users WHERE uname
like '%{self.see_id.text()}%' AND uid > 0")

        rows = self.cursor.fetchall()

        self.id_table.setRowCount(len(rows))
        for i, row in enumerate(rows):
            for j, col in enumerate(row):
                self.id_table.setItem(i, j,
QTableWidgetItem(str(col)))
            except Exception as e:
                traceback.print_exc()
                QtWidgets.QMessageBox.warning(self, "Table
Error", str(e))
            return

    def get_info(self):
        self.tid = self.id.currentText()

        query = "SELECT  uname,uaddress,email,password
FROM users WHERE uid = %s"
        self.cursor.execute(query, (self.tid,))
        result = self.cursor.fetchone()

        try:
            self.name.setText(result[0])
            self.add.setText(result[1].capitalize())
            self.ueail.setText(result[2])
            self.upass.setText(result[3])

        except Exception as e:
            traceback.print_exc()
            QtWidgets.QMessageBox.warning(self,
"Uploading Error", str(e))
            return

    def update_emp(self):
        try:
            id = self.id.currentText()

```

```

        name = self.name.text()
        address = self.add.text()
        email = self.uemail.text()
        password = self.upass.text()

        query = 'UPDATE users SET uname = %s,
uaddress = %s, email = %s, password = %s WHERE uid = %s'
        self.cursor.execute(query, (name, address,
email, password, int(id)))
        self.db.commit()
        self.id.setCurrentText(" ")
        self.name.clear()
        self.add.clear()
        self.uemail.clear()
        self.upass.clear()

        query = "SELECT
uid,uname,uaddress,email,password FROM users WHERE
uid>0"

        self.cursor.execute(query)

        rows = self.cursor.fetchall()

        self.id_table.setRowCount(len(rows))
        self.id_table.setColumnCount(len(rows[0]))

        for i, row in enumerate(rows):
            for j, col in enumerate(row):
                self.id_table.setItem(i, j,
QTableWidgetItem(str(col)))

        QtWidgets.QMessageBox.warning(self,
"UPDATE", "INFO UPDATED")
    except Exception as e:
        print(e)
        traceback.print_exc()

    def delete_emp(self):
        try:

```

```

        if self.id.currentText() == "":
            QtWidgets.QMessageBox.warning(self,
"ERROR", "ENTER ITEM ID")
            if self.id.currentText() != "":
                remove = "DELETE FROM users WHERE uid =
%s"
                self.cursor.execute(remove,
(self.tid,))
                self.db.commit()

                index = self.id.currentIndex()
                self.id.removeItem(index)
                self.id.setCurrentText(" ")

                self.name.clear()
                self.add.clear()
                self.uemail.clear()
                self.upass.clear()

                QtWidgets.QMessageBox.warning(self,
"REMOVED", "ITEM REMOVED")

                query = "SELECT
uid,uname,uaddress,email,password FROM users WHERE
uid>0"
                self.cursor.execute(query)

                rows = self.cursor.fetchall()

                self.id_table.setRowCount(len(rows))

self.id_table.setColumnCount(len(rows[0]))

                for i, row in enumerate(rows):
                    for j, col in enumerate(row):
                        self.id_table.setItem(i, j,
QTableWidgetItem(str(col)))

            except Exception as e:
                traceback.print_exc()

```

```

        QtWidgets.QMessageBox.warning(self, "Sql
Error", str(e))
        return

```

7. SALES REPORT

```

import os
import sys
import traceback
import matplotlib.pyplot as plt
from PyQt5.QtCore import QRect, Qt
from PyQt5.QtGui import QPixmap
from PyQt5 import QtWidgets, QtGui
from PyQt5.QtWidgets import QMainWindow, QHeaderView,
QTableWidget
import mysql.connector

def resource_path(relative_path):
    try:
        if getattr(sys, 'frozen', False):
            base_path = sys._MEIPASS
        else:
            base_path = os.path.dirname(__file__)

        resource_full_path = os.path.join(base_path,
relative_path)

        return resource_full_path
    except Exception as e:
        print(f"Error in resource_path: {e}")
        return relative_path

class sales(QMainWindow):
    def __init__(self):
        super().__init__()

```

```

        self.setWindowTitle("BILL")

        background_image =
resource_path('images/7.jpg')

        background_image_url =
background_image.replace("\\", "/")

        self.setStyleSheet(f'background-image:
url({background_image_url});')

        nextFont = QtGui.QFont()
        nextFont.setPointSize(20)
        nextFont.setFamily("Distant Galaxy")
        nextFont.setBold(True)

        label_font = QtGui.QFont()
        label_font.setPointSize(11)
        label_font.setFamily('18th Century')
        label_font.setBold(True)

        self.db = mysql.connector.connect(
            host="127.0.0.1",
            user="root",
            passwd="sairam",
            database="project"
        )
        self.cursor = self.db.cursor()

        self.title = QtWidgets.QLabel(self)
        self.title.setGeometry(530, 10, 380, 30)
        self.title.setText("SALES REPORT")
        self.title.setFont(nextFont)
        self.title.setStyleSheet("background:
transparent; color: white;")

        self.day = QtWidgets.QLabel(self)
        self.day.setGeometry(130, 60, 70, 30)
        self.day.setText("DAYS:")
        self.day.setFont(label_font)

```

```

        self.day.setStyleSheet("background:
transparent; color: white;")

        self.dname = QtWidgets.QComboBox(self)
        self.dname.setGeometry(180, 65, 161, 25)
        self.dname.setFont(label_font)
        self.dname.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")
        self.dname.addItem(
            ["ALL      DAYS",      "MONDAY",      "TUESDAY",
"WEDNESDAY",      "THURSDAY",      "FRIDAY",      "SATURDAY",
"SUNDAY"]])

        self.month = QtWidgets.QLabel(self)
        self.month.setGeometry(350, 60, 70, 30)
        self.month.setText("MONTH :")
        self.month.setFont(label_font)
        self.month.setStyleSheet("background:
transparent; color: white;")

        self.mname = QtWidgets.QComboBox(self)
        self.mname.setGeometry(420, 65, 161, 25)
        self.mname.setFont(label_font)
        self.mname.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")
        self.mname.addItem(
            ["JANUARY",  "FEBRUARY",  "MARCH",  "APRIL",
"MAY",  "JUNE",  "JULY",  "AUGUST",  "SEPTEMBER",  "OCTOBER",
"NOVEMBER",  "DECEMBER"]])

        self.year = QtWidgets.QLabel(self)
        self.year.setGeometry(590, 60, 70, 30)
        self.year.setText("YEAR:")
        self.year.setFont(label_font)
        self.year.setStyleSheet("background:
transparent; color: white;")

        self.yname = QtWidgets.QComboBox(self)

```

```

        self.yname.setGeometry(642, 65, 161, 25)
        self.yname.setFont(label_font)
        self.yname.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")
        self.yname.addItem(
            ["2024", "2025", "2026", "2027", "2028",
"2029", "2030", "2031", "2032"])

        self.cat = QtWidgets.QLabel(self)
        self.cat.setGeometry(810, 60, 90, 30)
        self.cat.setText("CATEGORY:")
        self.cat.setFont(label_font)
        self.cat.setStyleSheet("background:
transparent; color: white;")

        self.iname = QtWidgets.QComboBox(self)
        self.iname.setGeometry(903, 65, 161, 25)
        self.iname.setFont(label_font)
        self.iname.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")
        self.iname.addItem(
            ["ALL ITEMS", "PIZZA", "BURGER", "PASTA",
"MOMO", "NOODLES", "COMBOS", "DONUTS",
"COLD BEVERAGES", "HOT BEVERAGES"])

        self.back = QtWidgets.QPushButton(self)
        self.back.setText("BACK")
        self.back.setGeometry(QRect(1270, 10, 90, 30))
        self.back.setFont(label_font)
        self.back.clicked.connect(self.admin)
        self.back.setStyleSheet("color: white; border:
none;")

        self.confirm = QtWidgets.QPushButton(self)
        self.confirm.setText("CONFIRM")
        self.confirm.setGeometry(1080, 65, 100, 25)
        self.confirm.setFont(label_font)
        self.confirm.clicked.connect(self.report)

```

```

        self.confirm.setStyleSheet("color:          white;
border: r;")

        self.item_table = QtWidgets.QTableWidget(self)
        self.item_table.setColumnCount(3)

self.item_table.setHorizontalHeaderLabels(["ITEM NAME",
"QTY", "TOTAL"])
        self.item_table.setGeometry(60, 120, 422, 530)

self.item_table.setEditTriggers(QtWidgets.QAbstractItem
View.NoEditTriggers)

        self.item_table.setStyleSheet("""
                                QTableWidget {
                                background-color: transparent;
                                }
                                QHeaderView::section {
                                background-color: transparent;    /*
Make headers transparent */
                                color: white;                                /*
Change header text color to white */
                                font-weight: bold;                                /*
Make header text bold */
                                }
                                QTableWidget::item {
                                color: white;                                /*
Change item text color to white */
                                font-weight: bold;                                /* Make
item text normal weight */
                                }
                                QTableWidget::item[data-role="item"] {
                                font-weight: bold;                                /*
Make item name text bold if necessary */
                                }

                                """)

        self.item_table.setAttribute(Qt.WA_TranslucentBackgroun
d)

```

```

self.item_table.horizontalHeader().setSectionResizeMode
(0, QHeaderView.Stretch)

self.item_table.horizontalHeader().setSectionResizeMode
(1, QHeaderView.ResizeToContents)

self.item_table.horizontalHeader().setSectionResizeMode
(2, QHeaderView.ResizeToContents)

        self.graph_layout = QtWidgets.QLabel(self)
        self.graph_layout.setGeometry(590, 120, 0, 0)
        self.graph_layout.setStyleSheet(
            "background: transparent; border: 1px solid
            rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.showMaximized()

    def report(self):
        if self.iname.currentText() != "ALL ITEMS":
            if self.dname.currentText() == "ALL DAYS":
                self.monthly()
            else:
                self.weekly()
        if self.iname.currentText() == "ALL ITEMS":
            self.category_wise_sales()

    def category_wise_sales(self):
        try:
            month_index = ["01", "02", "03", "04", "05",
"06", "07", "08", "09", "10", "11", "12"]
            month
month_index[self.mname.currentIndex()]
            year = self.yname.currentText()
            day = self.dname.currentText()
            value = str(year) + "-" + str(month)

            if day == "ALL DAYS":
                content = ""

```

```

        SELECT  i.category AS Category,
SUM(f.qty) AS Quantity, SUM(f.qty * f.total) AS Total
        FROM food_txn f
        JOIN  items  i  ON  f.item_name  =
i.iname

        WHERE LEFT(f.date_txn,7) = %s
        GROUP BY i.category
        ORDER BY SUM(f.qty) DESC
        """
        self.cursor.execute(content, (value,))
    else:
        content = """
        SELECT  i.category AS Category,
SUM(f.qty) AS Quantity, SUM(f.qty * f.total) AS Total
        FROM food_txn f
        JOIN  items  i  ON  f.item_name  =
i.iname

        WHERE LEFT(f.date_txn,7) = %s AND
DAYNAME(f.date_txn) = %s
        GROUP BY i.category
        ORDER BY SUM(f.qty) DESC
        """
        self.cursor.execute(content, (value,
day,))

    rows = self.cursor.fetchall()

    self.item_table.setRowCount(len(rows))

self.item_table.setColumnCount(len(rows[0]))

    for i, row in enumerate(rows):
        for j, col in enumerate(row):
            self.item_table.setItem(i, j,
QTableWidgetItem(str(col)))

    self.item_table.setRowCount(len(rows))

self.item_table.setColumnCount(len(rows[0]))

```

```

        for i, row in enumerate(rows):
            for j, col in enumerate(row):
                self.item_table.setItem(i, j,
QTableWidgetItem(str(col)))

self.item_table.horizontalHeader().setSectionResizeMode
(0, QHeaderView.Stretch)

self.item_table.horizontalHeader().setSectionResizeMode
(1, QHeaderView.ResizeToContents)

self.item_table.horizontalHeader().setSectionResizeMode
(2, QHeaderView.ResizeToContents)

        categories = []
        qty_sold = []

        for row in rows:
            categories.append(row[0])
            qty_sold.append(int(row[1]))

        plt.clf()
        plt.pie(qty_sold, labels=categories,
autopct='%1.1f%%', startangle=90)
        plt.title(f'SALES REPORT FOR CATEGORIES
({self.mname.currentText()}) {year}')

        file_name =
f'category_sales_{self.mname.currentText()}_{year}.png'
        plt.savefig(file_name, transparent=True)

        pixmap = QPixmap(file_name)

        if not pixmap.isNull():
            self.graph_layout.setPixmap(pixmap)

self.graph_layout.setFixedSize(pixmap.size())

```

```

self.graph_layout.setStyleSheet("border:      2px      solid
white; background: transparent;")
    else:
        self.graph_layout.setStyleSheet("")

        categories = []
        qty_sold = []

    except Exception as e:
        traceback.print_exc()
        self.graph_layout.clear()

self.graph_layout.setStyleSheet('background:
transparent')

        QtWidgets.QMessageBox.warning(self,      "No
content", str(e))
        return

    def monthly(self):
        try:
            month_index = ["01", "02", "03", "04", "05",
"06", "07", "08", "09", "10", "11", "12"]
            month
            =
month_index[self.mname.currentIndex()]
            year = self.yname.currentText()
            cat = self.iname.currentText()
            value = str(year) + "-" + str(month)

            content = "SELECT i.iname as Item,SUM(f.qty)
AS Quantity,SUM(f.qty * f.total) AS Total FROM food_txn
f JOIN items i ON f.item_name = i.iname WHERE
left(f.date_txn,7) = %s AND i.category = %s GROUP BY
i.iname ORDER BY SUM(f.qty) asc"
            self.cursor.execute(content, (value, cat,))
            rows = self.cursor.fetchall()

            self.item_table.setRowCount(len(rows))

```

```

self.item_table.setColumnCount(len(rows[0]))

    for i, row in enumerate(rows):
        for j, col in enumerate(row):
            self.item_table.setItem(i, j,
QTableWidgetItem(str(col)))

        self.item_table.setRowCount(len(rows))

self.item_table.setColumnCount(len(rows[0]))

    for i, row in enumerate(rows):
        for j, col in enumerate(row):
            self.item_table.setItem(i, j,
QTableWidgetItem(str(col)))

self.item_table.horizontalHeader().setSectionResizeMode
(0, QHeaderView.Stretch)

self.item_table.horizontalHeader().setSectionResizeMode
(1, QHeaderView.ResizeToContents)

self.item_table.horizontalHeader().setSectionResizeMode
(2, QHeaderView.ResizeToContents)

    self.items = []
    self.qty_sold = []

    row_count = self.item_table.rowCount()
    for row in range(row_count):
        item = self.item_table.item(row, 0)
        if item is not None:
            self.items.append(item.text())

    for row in range(row_count):
        item = self.item_table.item(row, 1)
        if item is not None:

```

```

self.qty_sold.append(int(item.text()))

        plt.clf()

        plt.pie(self.qty_sold,      labels=self.items,
autopct='%1.1f%%', startangle=90)
        plt.title(
            f'SALES      REPORT      FOR      THE      MONTH
{self.mname.currentText()},      {year}      FOR      ITEM
{self.iname.currentText().upper()}')

        file_name                                     =
f'{self.mname.currentText()}_{year}.png'

        plt.savefig(file_name, transparent=True)

        pixmap = QPixmap(file_name)

        if not pixmap.isNull():
            self.graph_layout.setPixmap(pixmap)

self.graph_layout.setFixedSize(pixmap.size())

self.graph_layout.setStyleSheet("border:      2px      solid
white; background: transparent;")

        else:
            self.graph_layout.setStyleSheet("")

        self.items = []
        self.qty_sold = []

    except Exception as e:
        traceback.print_exc()
        self.graph_layout.clear()

```

```

self.graph_layout.setStyleSheet('background:
transparent')

        QtWidgets.QMessageBox.warning(self,      "No
content", str(e))
        return

    def weekly(self):
        try:
            month_index = ["01", "02", "03", "04", "05",
"06", "07", "08", "09", "10", "11", "12"]
            month                                     =
month_index[self.mname.currentIndex()]
            year = self.yname.currentText()
            cat = self.iname.currentText()
            day = self.dname.currentText()
            value = str(year) + "-" + str(month)

            content = "SELECT i.iname as Item,SUM(f.qty)
AS Quantity,SUM(f.qty * f.total) AS Total FROM food_txn
f JOIN items i ON f.item_name = i.iname WHERE
left(f.date_txn,7) = %s AND DAYNAME(date_txn) = %s AND
i.category = %s GROUP BY i.iname ORDER BY SUM(f.qty) asc"
            self.cursor.execute(content, (value, day,
cat,))

            rows = self.cursor.fetchall()

            self.item_table.setRowCount(len(rows))

self.item_table.setColumnCount(len(rows[0]))

            for i, row in enumerate(rows):
                for j, col in enumerate(row):
                    self.item_table.setItem(i,      j,
QTableWidgetItem(str(col)))

            self.item_table.setRowCount(len(rows))

self.item_table.setColumnCount(len(rows[0]))

```

```

        for i, row in enumerate(rows):
            for j, col in enumerate(row):
                self.item_table.setItem(i, j,
QTableWidgetItem(str(col)))

self.item_table.horizontalHeader().setSectionResizeMode
(0, QHeaderView.Stretch)

self.item_table.horizontalHeader().setSectionResizeMode
(1, QHeaderView.ResizeToContents)

self.item_table.horizontalHeader().setSectionResizeMode
(2, QHeaderView.ResizeToContents)

        self.items = []
        self.qty_sold = []

        row_count = self.item_table.rowCount()
        for row in range(row_count):
            item = self.item_table.item(row, 0)
            if item is not None:
                self.items.append(item.text())

        for row in range(row_count):
            item = self.item_table.item(row, 1)
            if item is not None:
                self.qty_sold.append(item.text())

        plt.clf()

        plt.pie(self.qty_sold, labels=self.items,
autopct='%1.1f%%', startangle=45)
        plt.title(f'SALES REPORT FOR THE DAY
{self.dname.currentText()} FOR
{self.mname.currentText()}, {year}')

```

```

plt.savefig(f'{self.dname.currentText()}_{self.mname.currentText()}_{year}.png', transparent=True)

        pixmap = QPixmap(f'{self.dname.currentText()}_{self.mname.currentText()}_{year}.png')
        self.graph_layout.setPixmap(pixmap)

        if not pixmap.isNull():
            self.graph_layout.setPixmap(pixmap)

self.graph_layout.setFixedSize(pixmap.size())

self.graph_layout.setStyleSheet("border: 2px solid white; background: transparent;")

        else:
            self.graph_layout.setStyleSheet("")

            self.items = []
            self.qty_sold = []
        except Exception as e:
            traceback.print_exc()
            self.graph_layout.clear()

self.graph_layout.setStyleSheet('background: transparent')

        QtWidgets.QMessageBox.warning(self, "No items", str(e))
        return

    def admin(self):
        import admin_page
        self.s = admin_page.Admin()
        self.s.show()
        self.destroy()

```

8. VIEW BILL

```
import os
import sys
import traceback
from PyQt5 import QtWidgets, QtGui
from PyQt5.QtCore import Qt
from PyQt5.QtWidgets import QMainWindow, QHeaderView,
QTableWidgetItem
import mysql.connector

def resource_path(relative_path):
    try:
        if getattr(sys, 'frozen', False):
            base_path = sys._MEIPASS
        else:
            base_path = os.path.dirname(__file__)

        resource_full_path = os.path.join(base_path,
relative_path)

        return resource_full_path
    except Exception as e:
        print(f"Error in resource_path: {e}")
        return relative_path

class bill(QMainWindow):
    def __init__(self):
        super().__init__()

        self.setFixedWidth(600)
        self.setFixedHeight(620)

        self.setWindowTitle("BILL")
        background_image =
resource_path('images/8.jpg')
```

```

        background_image_url
background_image.replace("\\", "/")

        self.setStyleSheet(f'background-image:
url({background_image_url});')

    try:

        nextFont = QtGui.QFont()
        nextFont.setPointSize(20)
        nextFont.setFamily("Distant Galaxy")
        nextFont.setBold(True)

        label_font = QtGui.QFont()
        label_font.setPointSize(11)
        label_font.setFamily('18th Century')
        label_font.setBold(True)

        self.db = mysql.connector.connect(
            host="127.0.0.1",
            user="root",
            passwd="sairam",
            database="project"
        )
        self.cursor = self.db.cursor()

        self.title = QtWidgets.QLabel(self)
        self.title.setGeometry(235, 10, 140, 30)
        self.title.setText("SEE BILL")
        self.title.setFont(nextFont)
        self.title.setStyleSheet("background:
transparent; color: white;")

        self.id = QtWidgets.QLabel(self)
        self.id.setGeometry(20, 60, 70, 30)
        self.id.setText("O_ID :")
        self.id.setFont(label_font)
        self.id.setStyleSheet("background:
transparent; color: white;")

```

```

        self.num = QtWidgets.QLineEdit(self)
        self.num.setGeometry(80, 60, 70, 30)
        self.num.setFont(label_font)
        self.num.setStyleSheet(
            "background: transparent; border: 1px
solid rgba(255, 255, 255, 200); color: white; padding:
5px;")

        self.back = QtWidgets.QPushButton(self)
        self.back.setGeometry(500, 5, 75, 23)
        self.back.setText("BACK")
        self.back.setFont(label_font)
        self.back.clicked.connect(self.admin)
        self.back.setStyleSheet("background:
rgba(0, 0, 0, 150); color: white; border: none;")

        self.cust = QtWidgets.QLabel(self)
        self.cust.setGeometry(160, 60, 100, 30)
        self.cust.setText("NAME :")
        self.cust.setFont(label_font)
        self.cust.setStyleSheet("background:
transparent; color: white;")

        self.cust_v = QtWidgets.QLineEdit(self)
        self.cust_v.setGeometry(215, 60, 180, 30)
        self.cust_v.setFont(label_font)
        self.cust_v.setReadOnly(True)
        self.cust_v.setStyleSheet(
            "background: transparent; border: 1px
solid rgba(255, 255, 255, 200); color: white; padding:
5px;")

        self.phone = QtWidgets.QLabel(self)
        self.phone.setGeometry(400, 60, 100, 30)
        self.phone.setText("PHONE :")
        self.phone.setFont(label_font)
        self.phone.setStyleSheet("background:
transparent; color: white;")

        self.phone_v = QtWidgets.QLineEdit(self)

```

```

        self.phone_v.setGeometry(470, 60, 125, 30)
        self.phone_v.setFont(label_font)
        self.phone_v.setReadOnly(True)
        self.phone_v.setStyleSheet(
            "background: transparent; border: 1px
solid rgba(255, 255, 255, 200); color: white; padding:
5px;")

        self.date = QtWidgets.QLabel(self)
        self.date.setGeometry(20, 105, 120, 30)
        self.date.setText("DATE & TIME :")
        self.date.setFont(label_font)
        self.date.setStyleSheet("background:
transparent; color: white;")

        self.date_v = QtWidgets.QLineEdit(self)
        self.date_v.setGeometry(130, 105, 180, 30)
        self.date_v.setFont(label_font)
        self.date_v.setReadOnly(True)
        self.date_v.setStyleSheet(
            "background: transparent; border: 1px
solid rgba(255, 255, 255, 200); color: white; padding:
5px;")

        self.items = QtWidgets.QLabel(self)
        self.items.setGeometry(330, 105, 130, 30)
        self.items.setText("NO. OF ITEMS :")
        self.items.setFont(label_font)
        self.items.setStyleSheet("background:
transparent; color: white;")

        self.t_items = QtWidgets.QLineEdit(self)
        self.t_items.setGeometry(450, 105, 50, 30)
        self.t_items.setFont(label_font)
        self.t_items.setReadOnly(True)
        self.t_items.setStyleSheet(
            "background: transparent; border: 1px
solid rgba(255, 255, 255, 200); color: white; padding:
5px;")

```

```

        self.total = QtWidgets.QLabel(self)
        self.total.setGeometry(20, 150, 110, 30)
        self.total.setText("TOTAL AMT :")
        self.total.setFont(label_font)
        self.total.setStyleSheet("background:
transparent; color: white;")

        self.total_amt = QtWidgets.QLineEdit(self)
        self.total_amt.setGeometry(120, 150, 100,
30)

        self.total_amt.setFont(label_font)
        self.total_amt.setStyleSheet(
            "background: transparent; border: 1px
solid rgba(255, 255, 255, 200); color: white; padding:
5px;")

        self.total_amt.setReadOnly(True)

        self.confirm = QtWidgets.QPushButton(self)
        self.confirm.setText("CONFIRM")
        self.confirm.setGeometry(260, 150, 100, 30)
        self.confirm.setFont(label_font)
        self.confirm.clicked.connect(self.details)
        self.confirm.setStyleSheet("background:
rgba(0, 0, 0, 150); color: white; border: none;")

        self.bill_table =
QtWidgets.QTableWidget(self)
        self.bill_table.setColumnCount(4)

        self.bill_table.setHorizontalHeaderLabels(["ITEM NAME",
"QTY", "AMT", "TOTAL"])
        self.bill_table.setGeometry(120, 200, 345,
400)

        self.bill_table.setEditTriggers(QtWidgets.QAbstractItem
View.NoEditTriggers)

        self.bill_table.horizontalHeader().setSectionResizeMode
(QHeaderView.ResizeToContents)

```

```

self.bill_table.horizontalHeader().setSectionResizeMode
(0, QHeaderView.Stretch)

self.bill_table.horizontalHeader().setSectionResizeMode
(1, QHeaderView.ResizeToContents)

self.bill_table.horizontalHeader().setSectionResizeMode
(2, QHeaderView.ResizeToContents)

self.bill_table.horizontalHeader().setSectionResizeMode
(2, QHeaderView.ResizeToContents)

        self.bill_table.setStyleSheet("""
                                QTableWidget {
                                    background-color: transparent;
                                }
                                QHeaderView::section {
                                    background-color: transparent; /* Make
headers transparent */
                                    color: white; /* Change
header text color to white */
                                    font-weight: bold; /* Make
header text bold */
                                }
                                QTableWidget::item {
                                    color: white; /* Change
item text color to white */
                                    font-weight: bold; /* Make item
text normal weight */
                                }
                                QTableWidget::item[data-role="item"] {
                                    font-weight: bold; /* Make
item name text bold if necessary */
                                }
                                """)

self.bill_table.setAttribute(Qt.WA_TranslucentBackgroun
d)

        except Exception as e:

```

```

        traceback.print_exc()
        QtWidgets.QMessageBox.warning(self,
"Error", str(e))
        return

    def details(self):
        self.value = self.num.text()
        try:
            if not self.value:
                QtWidgets.QMessageBox.warning(self,
"Error", "PLEASE ENTER ORDER ID")
                return

            self.cursor.execute('SELECT      MAX(order_id)
FROM food_txn')
            value = self.cursor.fetchone()

            if int(self.value) <= (value[0]):
                try:
                    cust_name          =          "SELECT
distinct(cust_name) From food_txn WHERE order_id = %s"
                    self.cursor.execute(cust_name,
(self.value,))

                    result1 = self.cursor.fetchone()

                    if result1:
                        self.cust_v.setText(result1[0])

                    cust_num          =          "SELECT
distinct(cust_phone) From food_txn WHERE order_id = %s"
                    self.cursor.execute(cust_num,
(self.value,))

                    result2 = self.cursor.fetchone()

                    if result2:

self.phone_v.setText(result2[0])

                    date = "SELECT  distinct(date_txn)
From food_txn WHERE order_id = %s"

```

```

        self.cursor.execute(date,
(self.value,))

        result3 = self.cursor.fetchone()

        if result3:

self.date_v.setText(str(result3[0]))

        qty = "SELECT sum(qty) From food_txn
WHERE order_id = %s"
        self.cursor.execute(qty,
(self.value,))

        result4 = self.cursor.fetchone()

        if result4:

self.t_items.setText(str(int(result4[0])))

        amt = "SELECT sum(total) From
food_txn WHERE order_id = %s"
        self.cursor.execute(amt,
(self.value,))

        result5 = self.cursor.fetchone()

        if result5:

self.total_amt.setText(str(int(result5[0])))

        content = "SELECT item_name, qty,
price, total From food_txn WHERE order_id = %s"
        self.cursor.execute(content,
(self.value,))

        rows = self.cursor.fetchall()

self.bill_table.setRowCount(len(rows))

self.bill_table.setColumnCount(len(rows[0]))

        for i, row in enumerate(rows):

```

```

        for j, col in enumerate(row):
            self.bill_table.setItem(i,
j, QTableWidgetItem(str(col)))

        except Exception as e:
            traceback.print_exc()
            QtWidgets.QMessageBox.warning(self,
>Error", str(e))
            return
        else:
            QtWidgets.QMessageBox.warning(self,
>Error", "ORDER ID NOT FOUND")
            except Exception as e:
                traceback.print_exc()
                QtWidgets.QMessageBox.warning(self,
>Error", str(e))
            return

    def admin(self):
        import admin_page
        self.s = admin_page.Admin()
        self.s.show()
        self.destroy()

```

9. ADD EMPLOYEE

```
import os
import sys
import traceback
import mysql.connector
from PyQt5 import QtWidgets, QtGui, QtCore

def resource_path(relative_path):
    try:
        if getattr(sys, 'frozen', False):
            base_path = sys._MEIPASS
        else:
            base_path = os.path.dirname(__file__)

        resource_full_path = os.path.join(base_path,
relative_path)

        return resource_full_path
    except Exception as e:
        print(f"Error in resource_path: {e}")
        return relative_path

class signup(QtWidgets.QDialog):

    def __init__(self):
        super(signup, self).__init__()
        self.signUp()

    def signUp(self):

        self.setFixedWidth(400)
        self.setFixedHeight(500)

        background_image =
resource_path('images/4.jpg')

        background_image_url =
background_image.replace("\\", "/")
```

```

        self.setStyleSheet(f'background-image:
url({background_image_url});')

        label_font = QtGui.QFont()
        label_font.setPointSize(13)
        label_font.setFamily('18th Century')
        label_font.setBold(True)

        title_font = QtGui.QFont()
        title_font.setPointSize(21)
        title_font.setFamily("18th Century")
        title_font.setBold(True)

        headFont = QtGui.QFont()
        headFont.setPointSize(12)
        headFont.setFamily("18th Century")
        headFont.setBold(True)

        nextFont = QtGui.QFont()
        nextFont.setPointSize(12)
        nextFont.setFamily("18th Century")
        nextFont.setBold(True)

        tFont = QtGui.QFont()
        tFont.setPointSize(20)
        tFont.setFamily("Distant Galaxy")
        tFont.setBold(True)

        self.setWindowTitle("SIGNUP PAGE")

        try:
            self.db = mysql.connector.connect(
                host="127.0.0.1",
                user="root",
                passwd="sairam",
                database="project"
            )

            self.cursor = self.db.cursor()

```

```

        try:
            id_query = "select max(uid) from users"
            self.cursor.execute(id_query)
            re = self.cursor.fetchone()
            self.uid = int(re[0]) + 1
        except Exception as t:
            traceback.print_exc()
            QtWidgets.QMessageBox.warning(self,
"Sql Error", str(t))
            return
    except Exception as e:
        traceback.print_exc()
        QtWidgets.QMessageBox.warning(self, "Sql
Error", str(e))
        return
    self.head_le = QtWidgets.QLabel(self)
    self.head_le.setText("ADD MEMBER")
    self.head_le.setFont(headFont)
    self.head_le.setGeometry(110, 30, 350, 50)
    self.head_le.setStyleSheet("background:
transparent; color: white;")

    self.id_lbl = QtWidgets.QLabel(self)
    self.id_lbl.setFont(headFont)
    self.id_lbl.setText("EMP_ID")
    self.id_lbl.setGeometry(50, 100, 70, 30)
    self.id_lbl.setStyleSheet("background:
transparent; color: white;")
    self.id_le = QtWidgets.QLineEdit(self)
    self.id_le.setGeometry(230, 100, 150, 30)
    self.id_le.setFont(headFont)
    self.id_le.setReadOnly(True)
    self.id_le.setText(str(self.uid))
    self.id_le.setStyleSheet(
        "background: transparent; border: 1px solid
        rgba(255, 255, 255, 200); color: white; padding: 5px;")

    self.name_lbl = QtWidgets.QLabel(self)
    self.name_lbl.setFont(headFont)

```

```

        self.name_lbl.setText("NAME")
        self.name_lbl.setStyleSheet("background:
transparent; color: white;")
        self.name_lbl.setGeometry(50, 140, 70, 30)
        self.name_le = QtWidgets.QLineEdit(self)
        self.name_le.setGeometry(230, 140, 150, 30)
        self.name_le.setFont(headFont)
        self.name_le.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.gender_lbl = QtWidgets.QLabel(self)
        self.gender_lbl.setFont(headFont)
        self.gender_lbl.setText("GENDER")
        self.gender_lbl.setStyleSheet("background:
transparent; color: white;")
        self.gender_lbl.setGeometry(50, 180, 70, 30)
        self.gender_combo = QtWidgets.QComboBox(self)
        self.gender_combo.setGeometry(230, 180, 150, 30)
        self.gender_combo.setFont(headFont)
        self.gender_combo.addItem(["", "M", "F", "O"])
        self.gender_combo.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.phone_lbl = QtWidgets.QLabel(self)
        self.phone_lbl.setFont(headFont)
        self.phone_lbl.setText("PHONE")
        self.phone_lbl.setStyleSheet("background:
transparent; color: white;")
        self.phone_lbl.setGeometry(50, 220, 70, 30)
        self.phone_le = QtWidgets.QLineEdit(self)
        self.phone_le.setGeometry(230, 220, 150, 30)
        self.phone_le.setFont(headFont)
        self.phone_le.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.address_lbl = QtWidgets.QLabel(self)
        self.address_lbl.setFont(headFont)

```

```

        self.address_lbl.setText("ADDRESS")
        self.address_lbl.setStyleSheet("background:
transparent; color: white;")
        self.address_lbl.setGeometry(50, 260, 100, 30)
        self.address_le = QtWidgets.QLineEdit(self)
        self.address_le.setGeometry(230, 260, 150, 30)
        self.address_le.setFont(headFont)
        self.address_le.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.email_lbl = QtWidgets.QLabel(self)
        self.email_lbl.setFont(label_font)
        self.email_lbl.setText("EMAIL")
        self.email_lbl.setFont(headFont)
        self.email_lbl.setStyleSheet("background:
transparent; color: white;")
        self.email_lbl.setGeometry(50, 300, 70, 30)
        self.email_le = QtWidgets.QLineEdit(self)
        self.email_le.setGeometry(230, 300, 150, 30)
        self.email_le.setFont(headFont)
        self.email_le.setStyleSheet(
            "background: transparent; border: 1px solid
rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.password_lbl = QtWidgets.QLabel(self)
        self.password_lbl.setFont(label_font)
        self.password_lbl.setText("PASSWORD")
        self.password_lbl.setStyleSheet("background:
transparent; color: white;")
        self.password_lbl.setGeometry(50, 340, 95, 30)
        self.password_lbl.setFont(headFont)
        self.password_le = QtWidgets.QLineEdit(self)

        self.password_le.setEchoMode(QtWidgets.QLineEdit.Passwo
rd)
        self.password_le.setGeometry(230, 340, 150, 30)
        self.password_le.setFont(headFont)
        self.password_le.setStyleSheet(

```

```

        "background: transparent; border: 1px solid
        rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.show_password_checkbox =
        QtWidgets.QCheckBox("Show Password", self)

        self.show_password_checkbox.setStyleSheet("background:
        transparent; color: white;")
        self.show_password_checkbox.setGeometry(230,
        410, 150, 30)

        self.show_password_checkbox.stateChanged.connect(self.p
        assword_visibility)
        self.show_password_checkbox.setFont(headFont)

        self.confirm_password_lbl =
        QtWidgets.QLabel(self)
        self.confirm_password_lbl.setFont(label_font)
        self.confirm_password_lbl.setText("CONFIRM
        PASSWORD")

        self.confirm_password_lbl.setStyleSheet("background:
        transparent; color: white;")
        self.confirm_password_lbl.setGeometry(50, 380,
        180, 30)
        self.confirm_password_lbl.setFont(headFont)
        self.confirm_password_le =
        QtWidgets.QLineEdit(self)

        self.confirm_password_le.setEchoMode(QtWidgets.QLineEdi
        t.Password)
        self.confirm_password_le.setGeometry(230, 380,
        150, 30)
        self.confirm_password_le.setFont(headFont)
        self.confirm_password_le.setStyleSheet(
        "background: transparent; border: 1px solid
        rgba(255, 255, 255, 200); color: white; padding: 5px;")

        self.signup_btn = QtWidgets.QPushButton(self)
        self.signup_btn.setText("SIGN UP")

```

```

        self.signup_btn.setGeometry(150, 440, 100, 30)
        self.signup_btn.setFont(nextFont)

self.signup_btn.clicked.connect(self.checkAndSave)
        self.signup_btn.setStyleSheet("background:
rgba(0, 0, 0, 150); color: white; border: none;")

        self.back_btn = QtWidgets.QPushButton(self)
        self.back_btn.setText("BACK")
        self.back_btn.setGeometry(315, 5, 70, 30)
        self.back_btn.setFont(nextFont)
        self.back_btn.clicked.connect(self.admin)
        self.back_btn.setStyleSheet("background:
rgba(0, 0, 0, 150); color: white; border: none;")

    def checkAndSave(self):
        global uid
        name = self.name_le.text()
        gender = self.gender_combo.currentText().upper()
        phone = self.phone_le.text()
        address = self.address_le.text()
        email = self.email_le.text()
        password = self.password_le.text()

        if name == "" or gender == "" or phone == "" or
address == "" or email == "" or password == "":
            QtWidgets.QMessageBox.warning(self,
            "Please", "Fill the above information's first.")
            return

        if len(password) > 15:
            QtWidgets.QMessageBox.warning(self,
            "Password Limit Exceeded",
            "Password
length should not exceed 15 characters.")
            return

        if password != self.confirm_password_le.text():

```

```

        QtWidgets.QMessageBox.warning(self,
"Password Mismatch", "Passwords do not match.")
        return

        if gender not in ('M', 'F', "O"):
            QtWidgets.QMessageBox.warning(self,
"Invalid Gender", "Gender should be 'M' or 'O'.")
            return

        if len(phone) != 10:
            QtWidgets.QMessageBox.warning(self,
"Invalid Number", "Phone number not valid")
            return

        query = "INSERT INTO users (uid, uname, gender,
password, uaddress, email, uphone) VALUES (%s, %s, %s,
%s, %s, %s, %s)"
        values = (self.uid, name, gender, password,
address, email, phone)
        self.cursor.execute(query, values)
        QtWidgets.QMessageBox.information(self,
"Success", f"User successfully registered. ID is
{self.uid}")
        self.db.commit()

        self.uid += 1
        self.id_le.setText(str(self.uid))
        self.name_le.clear()
        self.gender_combo.setCurrentText("")
        self.phone_le.clear()
        self.address_le.clear()
        self.email_le.clear()
        self.password_le.clear()
        self.confirm_password_le.clear()

    def password_visibility(self, state):
        if state == QtCore.Qt.Checked:

self.password_le.setEchoMode(QtWidgets.QLineEdit.Normal
)

```

```
self.confirm_password_le.setEchoMode(QtWidgets.QLineEdit.  
Normal)  
    else:  
  
self.password_le.setEchoMode(QtWidgets.QLineEdit.Password)  
  
self.confirm_password_le.setEchoMode(QtWidgets.QLineEdit.  
Password)  
  
    def admin(self):  
        import admin_page  
        self.s = admin_page.Admin()  
        self.s.show()  
        self.destroy()
```

UI SCREENSHOTS

1. LOGIN PAGE

LOGIN PAGE

LOGIN

ID

PASSWORD

☐ SHOW PASSWORD

LOGIN

2. FOOD ORDERING SYSTEM

CATEGORY

- PIZZA
- BURGER
- PASTA
- MOMO
- NOODLES
- COMBOS
- DONUTS
- COLD BEVERAGES
- HOT BEVERAGES

MENU

Cosmic Crunch Pizza - ₹1100

A thin crust pizza topped with colorful bell peppers, red onions, mushrooms and a sprinkle of crunchy breadcrumbs.



Mediterranean Marvel Pizza - ₹1300

A round pizza with a golden crust, topped with creamy feta cheese, kalamata olives, sun-dried tomatoes, artichoke.



Garden Fiesta Pizza - ₹1000

A veggie-packed pizza featuring a whole wheat crust topped with vibrant cherry tomatoes, spinach, sweet corn, red bell peppers, and sliced jalapeños for a kick.

ORDER TABLE

	Item	Price	Quantity	Total	Remove
1	Mediterranean Marvel Pizza	1100	1	1100	REMOVE
2	Cosmic Crunch Pizza	1100	1	1100	REMOVE

Total Price: ₹2400.0

LOGOUT

Confirmation

?

×

DATE/TIME:

2024-11-20 12:16:48

NAME:

Sai

PHONE NO.:

1236547890

BACK

	Items	Qty	Rate
1	Mediterranean Marvel Pizza	1	1300.00
2	Cosmic Crunch Pizza	1	1100.00

ORDER NO.:

124

TOTAL:

2400.00

CGST/SGST:

120.00

DISCOUNT:

168.00

TOTAL AMT:

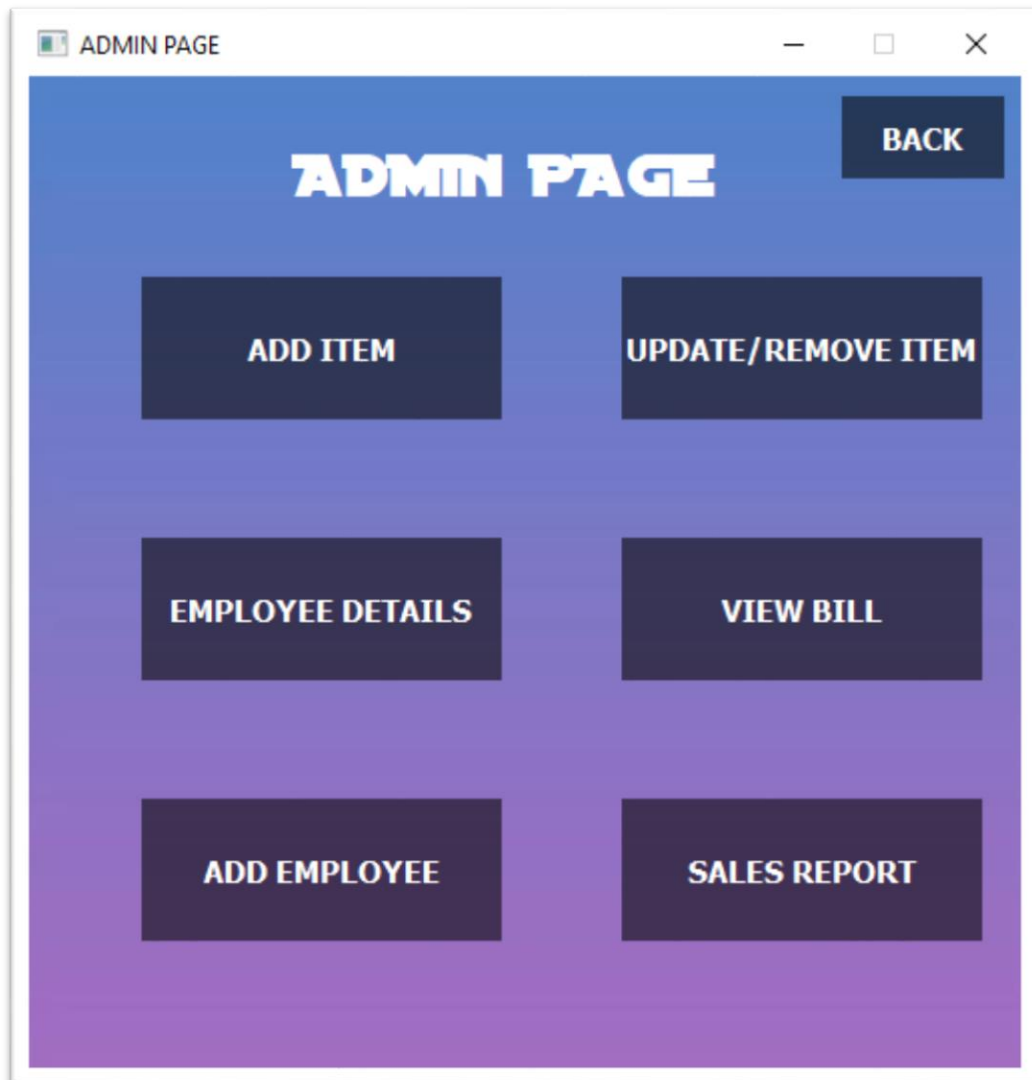
2352.00

NO. OF ITEMS:

2

CONFIRM

3. ADMIN PAGE



4. ADD ITEM

ADD ITEM

BACK

ADD ITEM

ITEM ID

91

CATEGORY

Pizza

ITEM NAME

Farmhouse Pizza

PRICE

1500

DESCRIPTION

Sample



UPLOAD IMAGE

ADD ITEM

5. REMOVE/UPDATE ITEM

UPDATE/REMOVE ITEM

BACK

REMOVE/UPDATE ITEM

ITEM ID

91

CATEGORY

Pizza

ITEM NAME

Farmhouse Pizza

PRICE

1500

DESCRIPTION

Sample1



UPLOAD IMAGE

UPDATE ITEM

REMOVE ITEM

SEARCH ITEM NAME

	ID	CATEGORY	ITEM	PRICE
1	1	Pizza	Cosmic Crunch Pizza	1100
2	2	Pizza	Mediterranean Marvel Pizza	1300
3	3	Pizza	Garden Fiesta Pizza	1000
4	4	Pizza	Spicy Samurai Pizza	1200
5	5	Pizza	Enchanted Forest Pizza	1300
6	6	Pizza	Zesty Zephyr Pizza	1000
7	7	Pizza	Smoky Mountain Meltdown Pizza	1400
8	8	Pizza	Tropical Tango Pizza	1100
9	9	Pizza	Firecracker Fusion Pizza	1300
10	10	Pizza	Maple Glaze Delight Pizza	1500

6. EMPLOYEE DETAILS

UPDATE/REMOVE ITEM

EMPLOYEE DETAILS

BACK

SEARCH EMPLOYEE NAME

	UID	NAME	ADDRESS	EMAIL	PASSWORD
1	1	Satyam	Ganesh bro	f	1
2	2	Sai	d	d	2
3	3	Sai	d	d	2
4	4	Pranava Bolo RK RK	Chennai	d	123
5	5	pranava	ptp	jjv	1234
6	6	pranava	ptp	jjv	1234
7	7	satyam	mp	f	1
8	8	f	erg	gr	1
9	9	rndom	ff	fr	2
10	10	lala	d	f	1

EMPLOYEE ID

6

NAME

pranava

ADDRESS

Ptp

EMAIL

jjv

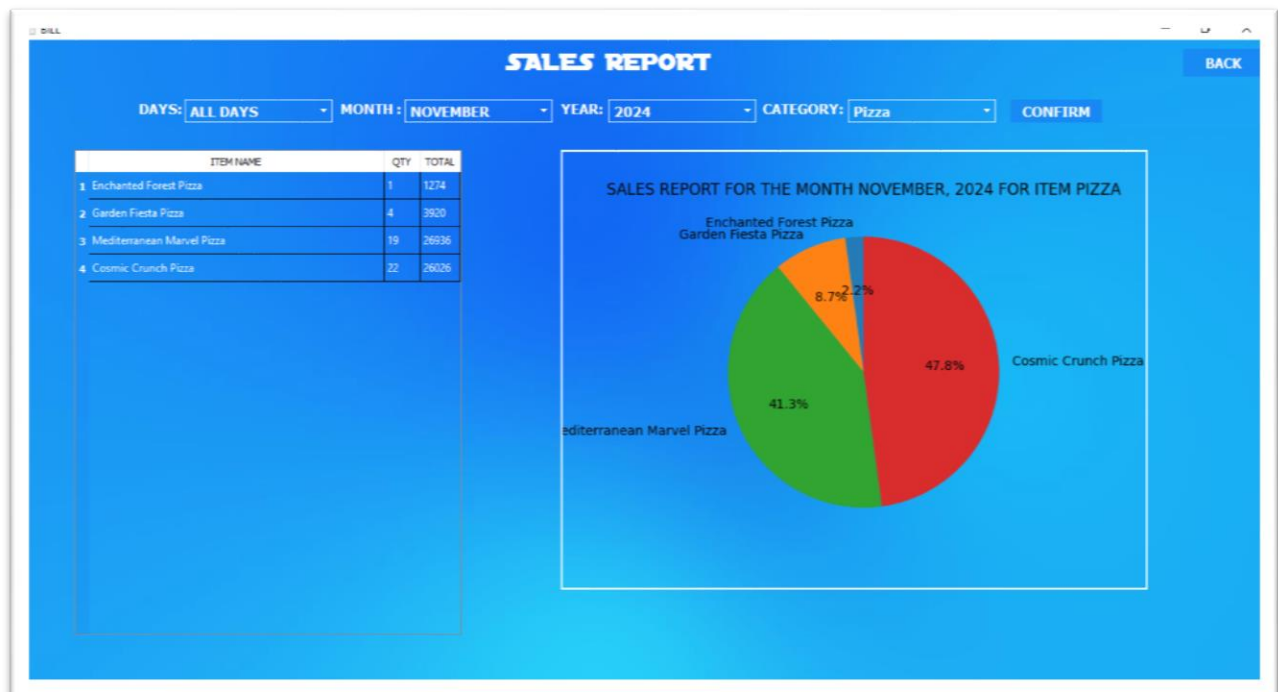
PASSWORD

1234

UPDATE ITEM

REMOVE ITEM

7. SALES REPORT



8. VIEW BILL

BILL

SEE BILL

BACK

O_ID : 124

NAME : Sai

PHONE : 1236547890

DATE & TIME : 2024-11-20 12:26:05

NO. OF ITEMS : 2

TOTAL AMT : 2352

CONFIRM

	ITEM NAME	QTY	AMT	TOTAL
1	Cosmic Crunch Pizza	1	1100	1078
2	Mediterranean Marvel Pizza	1	1300	1274

9. ADD EMPLOYEE

SIGNUP PAGE

?

×

BACK

ADD MEMBER

EMP_ID	14
NAME	Sai
GENDER	M
PHONE	1234567890
ADDRESS	ptp
EMAIL	sai@gmail.com
PASSWORD	•
CONFIRM PASSWORD	•
	☐ Show Password

SIGN UP

FUTURE ENHANCEMENTS

This application can be enhanced in several ways to improve its functionality and user experience:

- ✚ **Order Confirmation via SMS:** Send a confirmation message to the customer's phone upon placing an order, providing immediate assurance.
- ✚ **Automatic Bill Generation:** Configure the system to automatically generate a bill once an order is placed, streamlining the billing process and reducing errors.
- ✚ **Customer Ordering Interface:** Allow customers to place orders directly through the software, enhancing the dining experience by reducing wait times and enabling menu browsing and order customization.
- ✚ **Responsive Design:** Implement a dynamic screen size to ensure the application adjusts seamlessly to various devices, from desktops to smartphones.
- ✚ **Add-On Options:** Provide customers with the ability to select add-ons for each item, offering more choices and personalized orders.

BIBLIOGRAPHY

-  NCERT INFORMATICS PRACTICES TEXT BOOK
-  www.stackoverflow.com
-  www.geeksforgeeks.org
-  www.qtdocumentation.com
-  www.microsoftdesigner.com
-  www.magicstudios.com
-  www.youtube.com